



Софийски университет „Св. Кл. Охридски“
Факултет по математика и информатика
Бакалавърска програма
„Софтуерно инженерство“

КУРСОВА РАБОТА

специалност „Софтуерно инженерство“

Тема: Стратегическа игра за защитаване на база,
реализирана посредством Rust и Bevy

Ъ

Изработил:

Илиян Гаврилов Гаврилов

СОФИЯ

2 0 2 3

УВОД

Първите видеоигри се появяват в средата на миналия век. Те са били доста прости и са се разработвали само от учени на специални компютри, които не са били широко разпространени.

От 70-те години на XX век започва разпространението на първите аркадни видео игри. Някои от най-популярните са Pong, Space Invaders, Donkey Kong и други.

Чак към края на XX век започват да се разпространяват персонални компютри, които всеки може да закупи. Развиват се и технологиите, използвани в компютрите, и вследствие на това се развиват и видео игрите.

Първата игра от тип „защитна кула“ (tower defense) излиза през 1990 г. с името Rampart, изработена от Atari Games и е аркадна игра. Счита се за основа на жанра.

„Защитна кула“ е поджанр на стратегически игри, в които играчът трябва да защити свои територии, (най-често някаква база/кула) като възпрепятства враговете. Противниците се движат по определена пътека и играчът трябва да ги спре преди те да достигнат края на пътеката, като строи и подобрява различни видове защитни структури (кули). Кулите могат да забавят, блокират или атакуват враговете. Целта на играча е да преодолее възможно най-много вълни от чудовища без да позволи на противниците да стигнат до края на пътя. Жанрът “защитна кула” по принцип е поджанр на „стратегия в реално време“ (Real-time strategy) видео игри, но някои по-нови и по-модерни игри имплементират аспекти на походови (turn-based) стратегии. [1, 2, 3]

Игрите от такъв тип изискват добре развито стратегическо мислене. Съществени са елементите на управляването на ресурсите, поради ограничения им набор, стратегическото позициониране на кулите, планиране, противодействие на различни видове врагове с помощта на подходящи кули и т.н.

Защо се играят игри от типа „защитна кула“? Един от биологичните инстинкти на хората е да защитават това, за което се грижат. Игрите от жанра „защитна кула“ са базирани точно на тези вградени в хората инстинкти. Те ни дават база и ние по интуиция ще се опитаме да я защитим, независимо по какъв начин. [4]

Целта на настоящата дипломна работа е да се разработи двуизмерна игра от тип „защитна база“ за защита на база от противници, чрез строенето на различни отбранителни структури, реализирана посредством Rust и Bevy.

ПЪРВА ГЛАВА

ИСТОРИЯ НА ЖАНРА ЗАЩИТНА КУЛА. ПРЕГЛЕД И ПРОУЧВАНЕ НА ДРУГИ ИГРИ ОТ ЖАНРА. МЕТОДИ И ТЕХНОЛОГИИ ЗА ТЯХНАТА РЕАЛИЗАЦИЯ

1.1 История на игри от типа „защитна кула“

1.1.1 Ранна история – аркадни игри

Историята на игрите „защитна кула“ започва още през 80-те години на миналия век. Може да се каже, че първите предшественици на жанра „защитна кула“ са аркадните игри. Тези игри по-скоро се смятат за защитни (defense) игри, но дават вдъхновение на модерните игри в жанра.

В Space Invaders (1978 г.) целта е да се защити територията на играча от множество противници. Играта включва щитове, които възпрепятстват атаките на противниците и помагат на играча да защити териториите си. Все още няма имплементирана функционалността за строеж на защитни кули, а вместо това играчът сам атакува враговете си.

Missile Command (1980 г.) следва подобно развитие, като в нея противниците могат да се движат в няколко посоки. Но в тези игри все още не е бил добавен компонентът за построяване на защитни кули и враговете се движат само на посоки, а не по определена пътека. [5]

В Defender (1981 г.) и Choplifter (1982 г.) започва тенденция към защитаване на предмети. Излизат още много популярни аркадни игри, които развиват идеята за „defense“ игри – Star Wars: The Empire Strikes Back

(1982 г.), Sorcerer's Apprentice (1983 г.), Vermin (1980 г.), Manhole (1981 г.), Parachute (1981 г.), Popeye (1981 г.), Greenhouse (1982 г.) и т.н.

През средата на 80-те години продължават да еволюират стратегическите елементи и излизат първите игри за персонални компютри – Gandalf the Sorcerer (1984 г.) и Pedro (1984 г.). [6]

Rampart (1990 г.) се счита за първата игра от жанра „защитна кула“. В нея играчът може да строи защитни структури, които автоматично атакуват противниците. В нея са налични елементи на строене, защитаване и поправяне на самата база. [7, 8]

Две години по-късно излиза и Dune II: The Building of a Dynasty, която също се счита за основна в жанра. Тя включва строежа на база и управление на ресурси, които по-късно стават опорни точки в жанра. [9, 10]

След широкото разпространяване на компютърната мишка излизат още игри, които доразвиват жанра – Ambush at Sorinor (1993 г.), Fort Condor мини игра в Final Fantasy VII (1997 г.), Attack of the Mutant Penguins (1995 г.), Dungeon Keeper (1997 г.), Turok 2: Seeds of Evil (1998 г.), Fortress (2001 г.). През 2006 г. излизат и 2 мини игри в Warcraft III (2002 г.) – Element TD и Gem Tower Defense, които са първите усвояващи името “Tower Defense” или съкращението TD. [7, 8, 11, 12, 13]

1.1.2 Периодът от 2007 до днес

С възхода на Flash игрите през 2007 г. излизат и първите самостоятелни браузърни игри – Flash Element Tower Defense, Desktop Tower Defense, в която играчът построява пътеката на враговете чрез позиционирането на защитните си кули, и Antbuster. Излизат още няколко

популярни в жанра заглавия като GemCraft (2008 г.), Bloons Tower Defense (2007 г.) и Plants vs. Zombies (2009 г.). [14, 15, 16, 17, 18, 19, 20, 21]

Следват и още много игри през „бума“ на 2007 – 2010 г. Огромният успех на жанра довежда до множество най-различни игри. Игрите еволюират от top-down перспектива (поставяне на гледната точка с изглед отгоре). Излизат първите триизмерни и first-person (поставяне на гледната точка от първо лице) игри – Iron Grip: Warlord (2008 г.), Sanctum (2011 г.). Dungeon Defenders (2010 г.) добавя third-person перспектива (поставяне на гледната точка от трето лице). [22, 23]

Някои от по-скорошните популярни издания включват Bloons TD 6 (2018 г.), Tower Defense Simulator в Roblox (2019 г.) и Taur (2020 г.).

1.2 Преглед и проучване на други съвременни игри от жанра

1.2.1 Bloons TD 6



Фиг. 1.1 Bloons TD 6

Bloons TD 6 (BTD 6) е игра от тип „защитна кула“, изработена и публикувана от Ninja Kiwi през декември 2018 г. Тя е шестата поредна игра в „Bloons TD“ серията, която добавя много нови неща и подобрения в графиката. [24]

Играчът трябва да поставя кули на стратегически места и да използва специалните им свойства, за да се защити от вълни от балони, които се движат по определена пътека. Според цвета на балона, след като бъде спукан, може да се появи друг балон, докато не се спуква най-низшият балон – червеният. Играта включва голямо разнообразие от кули с множество различни подобрения, всяка с нейните уникални способности. Разработена е с игровия двигател Unity. [24]

В играта също има и разнообразие от режими: “стандартен режим”, където играчът трябва да победи колкото може повече вълни от балони, както и „предизвикателен режим”, където играчът трябва да изпълни определени мисии и цели. [24]



Фиг. 1.2 Снимка от играта

Играта има няколко важни и иновативни идеи в жанра.

Както в много други игри от жанра, и в тази игра има множество готови карти с предварително зададени пътеки и избор от няколко трудности. Картите се разделят по трудност на “начинаещи“, „междинни“, „напреднали“ и „експертни“. Преди започването на играта, играчът може да избере режим на играта – лесен, междинен или труден, като всеки един от тях си има и други специализирани режими към него (фиг. 1.3). Например в лесния режим има опция за “Primary Monkeys Only”, което означава, че играчът може да използва само маймуни от началния клас. Има общо 17 режима. За всеки спечелен режим (ниво) играчът е награден с определен медал за картата. [25, 26]



Фиг. 1.3 Трудни режими в Bloons TD 6

В нея освен кули са добавени и така наречените „герои“, които също помагат на играча по един или друг начин. Тези герои се отключват с пари. Когато бъдат поставени, те започват постепенно да събират опит и отключват нови умения, автоматично си подобряват статистиките, като

колкото повече време стоят на полето, толкова по-силни стават. Те също се различават от нормалните кули по това, че може да се сложи само един единствен герой на ниво. [27]

Кулите се отключват постепенно докато се вдигат нива в играта. Когато играчът вдигне ниво играта му предоставя избор от няколко кули и той може да избере най-подходящата, която да използва. Самите кули също така събират опит, докато са поставени на полето и според това колко време са били в битка, могат да отключват нови подобрения. Тези подобрения се разделят в 3 пътеки по 5 подобрения във всяка пътека (фиг. 1.4). Всяка пътека предоставя различни подобрения на кулата. Играчът може да закупи всички подобрения до последния в една пътека и след това може да купи до 2 от една от останалите пътеки. Тоест когато се купят 2 подобрения от 2 различни пътеки, 3-тата се заключва и не може да се използва на съответната кула. Кулите и техните подобрения се купуват с пари, които играчът печели от всеки спукан балон и след всяка победена вълна от балони. [28]



Фиг. 1.4 Пътеки с ънгрейди в Bloons TD 6

Някои от кулите имат последен свръхмощен ъпгрейд, наречен „paragon”, който изисква жертването на 3 кули, които имат трите последни ъпгрейди от всички пътеки. [29]

Друга уникална идея на тази игра е „Monkey Knowledge” (фиг. 1.5). Това е система, която позволява на играча да отключва постоянни подобрения за своите кули. Тази система се състои от 6 различни ъпгрейд дървета за всеки вид маймуна, едно за героите и едно за специалните умения. Тази система награждава играча постепенно, докато той играе различни нива и го подтиква да играе още докато не отключи всички подобрения в дърветата. На всяко вдигане на ниво след 30-то играчът е награден с един „Monkey Knowledge Point”, който може да използва, за да купи ъпгрейд от дървото. [30]



Фиг. 1.5 “Monkey Knowledge” дърво в Bloons TD 6

В играта има и multiplayer режим, в който се разделят парите, а понякога и полето, поравно между играчите и всеки контролира своите кули и играчите могат да изпращат пари един на друг. [31]

Играта също има свой собствен редактор на нива и с него всеки може да прави различни предизвикателства и да ги споделя с другите. [32]

1.2.2 Tower Defense Simulator в Roblox



Фиг. 1.6 Tower Defense Simulator в Roblox

Tower Defense Simulator е триизмерна игра, разработена и публикувана от Paradoxum Games през юни 2019 г. Играта позволява на няколко играчи да се бият с вълни от различни врагове, докато не загубят или победят конкретната карта. Играчите печелят пари, като нанасят щети на враговете и от бонуси за вълни, които впоследствие могат да бъдат инвестирани в закупуване на нови кули или надграждане на съществуващи. След като завършат картата, те получават опит (EXP), с който повишават нивото си и отключват нови кули, и монети, или скъпоценни камъни, които могат да се използват за закупуване на нови кули. [33]

В тази игра нови кули се отключват с пари или скъпоценни камъни, които се дават след всяка игра. В играта играчът може да си избере до 5 кули, които да ползва, преди да започне играта. [33]

За разлика от Bloons TD 6 противниците умират след като бъдат убити и не се появяват други по-слаби противници на тяхно място.

В Tower Defense Simulator има само 4 режима на игра – “normal”, “molten”, “golden” и “fallen”. За разлика от Bloons TD 6 на всеки режим се започва от първата вълна и се завършва на 40-тата (с изключение на режим „normal”, който завършва на 30-тата) и няма “freerplay”, тоест играчът трябва да се върне в лобито и не може да продължи да играе. Във всеки режим си има специфични противници и на последната вълна винаги има финално бос чудовище, което има много кръв и специални умения. „Golden” режимът е вече премахнат, но в него специалните златни противници не могат да бъдат убити от експлозии. [34]



Фиг. 1.7 Снимка от играта

Играта е триизмерна и за разлика от Bloons TD 6 има кули, които могат да се поставят само на издигнати терени като планини, сгради и т.н. Един от недостатъците на играта е, че кулите могат да стрелят през други

обекти, които се явяват между кулата и врага, което не е много реалистично. В Bloons, макар и да е двуизмерна игра, има система за линия на видимост и когато някоя кула е поставена до някакво препятствие тя не може да вижда и атакува противниците през него.

Повечето кули в играта са офанзивни, но има и няколко които подкрепят другите кули. Например има лекар, който може да възстановява кули, които са били зашеметени от някое умение на противника. Има диджей, който дава отстъпки на ъпгрейдите на кулите и генерал, който подсилва атаката на кулите. [35]

Играта също така предлага и още един вид кули, които не само стрелят, но се движат по пътеката в обратната посока и на всяко блъскане с враг, нанасят щета върху него. Те се наричат "units" и по същество са мобилни превозни средства или хора, които се движат по пътя от изхода до входа. Някои от тях са неподвижни. Те обикновено се появяват през определен интервал от време, но някои изискват намеса на играча – да активира някакво умение или да изпълни специфично изискване. В двупосочните карти те се пускат по случаен път. [36]

В тази игра има специални кули, които се отключват като награди на 30, 50, 75 и 100-но ниво, което подтиква играча да продължава да играе, но не е толкова ефективно колкото „Monkey Knowledge” в Bloons TD 6. [35]

Играта е направена чрез игровия двигател на Roblox и със скриптовия език за програмиране Lua. [33]

1.2.3 GemCraft



Фиг. 1.8 GemCraft

GemCraft е публикувана от „Game in a Bottle” през 2008 г. Тя притежава елементите на класическата игра от тип „защитна кула“, където играчът трябва да се защитава срещу вълни от врагове чрез стратегическо поставяне на кули и използване на специални способности. Играта включва разнообразие от кули и надстройки, всяка с уникални способности и силни страни. [37, 38]

Играта разполага с уникална функционалност, която позволява на играчите да създават и комбинират скъпоценни камъни, за да захванват своите кули и магии. [39]



Фиг. 1.9 Снимка от играта

Друг отличителен белег е липсата на пари. Вместо пари играчът събира „мана“ в базата си, която едновременно служи като негова кръв и валута. Тоест това добавя още един елемент към стратегията, където играчът трябва да помисли колко точки от живота си иска да жертва, за да построи някоя нова кула, или за да подобри статистиките на вече поставена кула, и едновременно с това да съобрази факта, че ако прекалено много врагове достигнат базата му, той ще загуби играта. [39]

1.2.4 Rogue Tower



Фиг. 1.10 Rogue Tower

Rogue Tower е игра от тип „защитна кула“ с rogue-подобни елементи и непрекъснато разширяващ се път. Играта е сравнително нова – публикувана е през януари 2022 г. Rogue-подобните игри са известни със своите произволно генерирани нива и походов „геймплей“, в който играчът започва всяка следваща вълна (ниво) след натискането на бутон. В тази игра първоначалните статистики на кулите зависят от височината, на която са поставени. Ъпгрейдите на кулите се правят чрез теглене на карти и пари. След всяко разширение на картата, започва следващия рунд. Разширението на картата разкрива нова случайна зона, в която може да има ресурси, къщи или други постройки. [40, 41, 42]

За разлика от другите игри от тип „защитна кула“ в тази няма готови карти. Картата се изгражда чрез произволно генерирани плочки от терен. Тези плочки се генерират от играча след натискането на бутон. Играчът има избор в коя посока да разшири терена. [42]



Фиг. 1.11 Снимка от играта

В тази игра също има дърво с прогрес, както в Bloons TD 6. Колкото повече време вложи играчът в играта, толкова по-силни му стават защитите. [43]

1.3 Методи за създаване на видео игри

Методът, използван за създаване на видео игра, зависи от целите, ресурсите и опита на разработчика.

1.3.1 Разработка на игра с игрови двигател

Игровите двигатели предоставят предварително изградена среда за създаване на видео игри. Игровият двигател позволява на програмистите

да се фокусират върху изработването на съдържанието и механиката на играта, без да се интересуват от програмиране на ниско ниво. Чрез него програмистите могат по-лесно да разработват играта чрез абстрактните интерфейси, които игровият двигател предоставя. Той има вградени библиотеки, инструменти и графичен интерфейс, което го прави лесно използваем. Този метод е най-предпочитан поради големите абстракции, които опростяват разработката на видео игрите.

1.3.2 Разработка на игра с персонализиран игрови двигател

Някои разработчици на игри избират да създадат свой собствен персонализиран игрови двигател. Това им позволява да моделират игрите си по-лесно и по-гъвкаво. Игровият двигател може постоянно да търпи промени според нуждите на екипа в хода на разработка. Този подход обаче изисква много време и се ползва предимно от големи компании, които инвестират много време и ресурси и организират цели екипи, които да изградят и поддържат такъв игрови двигател.

1.3.3 Разработка на игра без игрови двигател

Програмистите могат да използват директно езици като C, C++, C#, Java, Python и други, с които да разработват игра. Изисква се разбиране в множество сфери на програмирането, тъй като с този подход трябва програмистът сам да си изгради библиотеки за графика, звук, анимации и други, което може да се окаже доста продължителен процес. Този метод е по-комплексен и изисква широки знания в множество специализирани сфери, но дава повече контрол върху производителността и функционалността на играта.

Най-често използвани готови библиотеки при този подход са:

- библиотеки за графика (създаване и изобразяване на висококачествена графика) – DirectX, OpenGL, и Vulkan
- библиотеки за физика (симулиране на реалистични физики в игрите като динамика, анимация и засичане на колизии и сблъсъци) – Havok, Bullet, PhysX
- библиотеки за аудио (създаване и възпроизвеждане на звукови ефекти и музика в игрите) – FMOD, Wwise, OpenAL

1.3.4 Разработка на игра с комплекти за разработка на игри

Тези комплекти представляват специализиран хардуер и софтуер, използван за създаването на видео игри. Те предоставят опростен начин за създаване на игра. Предлагат графичен интерфейс, където може да се създават нива, герои и други елементи на играта и най-често се използват от начинаещи. За разлика от игровите двигатели, тези игрови комплекти са по-достъпни и по-лесни за ползване, но дават по-малко свобода на разработчика и по-лимитиран избор от функционалности.

Най-често използваните комплекти са:

- RPG Maker
- Construct
- GameMaker Studio

1.3.5 Хибриден подход

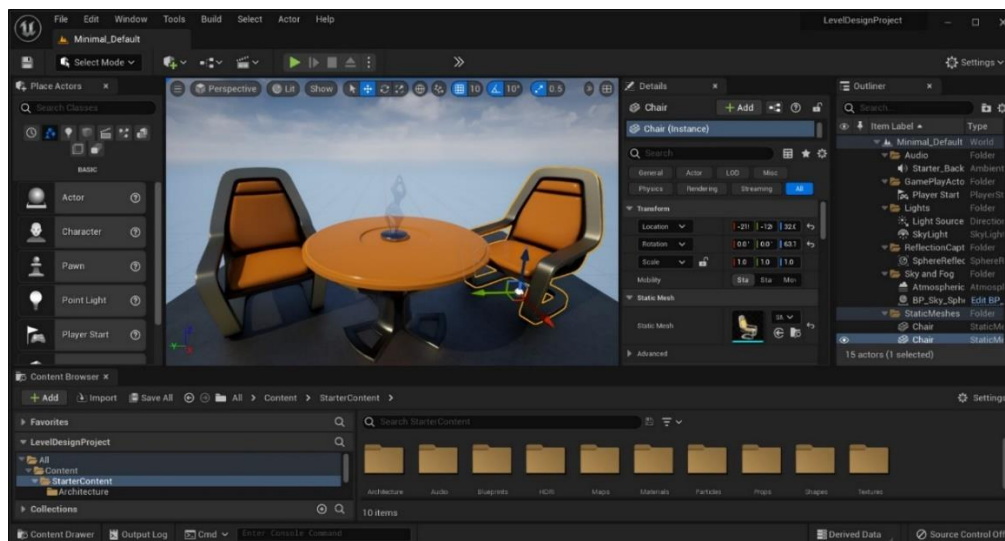
Някои разработчици на игри използват комбинация от различни методи, за да създават игри. Например може да ползват игрови двигател, за да управляват определени аспекти на играта, като например физиката и

изобразяването, и едновременно да използват други езици за програмиране, за да управляват други части на играта, като например изкуствения интелект.

1.4 Съществуващи софтуерни продукти и технологии за създаване на видео игри

1.4.1 Unreal Engine

Unreal Engine е триизмерен игрови двигател, създаден от Epic Games през 1998 г. Последната версия (Unreal Engine 5) излиза през май 2020 г. и предоставя „Nanite” виртуализирана геометрия и глобално осветление „Lumen“, които правят графиките още по-реалистични. Представява голям набор от инструменти и библиотеки за разработване на игри, включително игрови двигател, визуален редактор, двигател за физика, система за анимация и много други. Това е много мощен двигател за игри, който се използва за създаването на много висококачествени и визуално зашеметяващи игри. Той е известен със своите напреднали и усъвършенствани графични възможности и изобразяване в реално време. С този двигател разработчиците успяват да създадат много реалистични обстановки в своите игри чрез разнообразни функционалности като динамичната светлина. Игровият двигател е способен да обработва милиони обекти едновременно. [44, 45, 46, 47]



Фиг. 1.12 Интерфейс на Unreal Engine

Unreal Engine е междуплатформен двигател, което означава, че игрите, разработени с него, могат да се играят на множество различни платформи, включително персонални компютри, мобилни устройства, конзоли и VR устройства (устройства за виртуална реалност). [48]

Двигателят ползва „Blueprint” система, служеща за визуално програмиране. Тя дава възможност дори на хора, които не се занимават с програмиране, да изработват собствени игри. Работи на базата на свързване на блокове един с друг според определена логическа връзка. [49]

Недостатък на Unreal Engine е, че не поддържа разработването на игри за по-стари конзоли.

Той е безплатен игрови двигател, но след като някоя игра започне да печели над 1 милион долара, Epic Games започват да взимат 5 % от приходите на играта – били те от продажбата на играта, вградени покупки, реклами или други. [50]

Двигателят също така поддържа широк набор от езици за програмиране, включително C++, C# и Python, което позволява на разработчиците да създават персонализирани механики на играта, системи с изкуствен интелект и други функционалности.

1.4.2 Unity

Unity е един от най-използваните игрови двигатели. Той е междуплатформен двигател, който поддържа разработката на игри за различни платформи, включващи персонални компютри, конзоли и мобилни устройства. Известен е с улеснения си интерфейс и с широкия си набор от функционалности, които включват визуален редактор, физически двигател и поддръжка за множество езици за програмиране. Разработен е от Unity Technologies през 2005 г. Unity използва ECS системата. [51]



Фиг. 1.13 Интерфейс на Unity

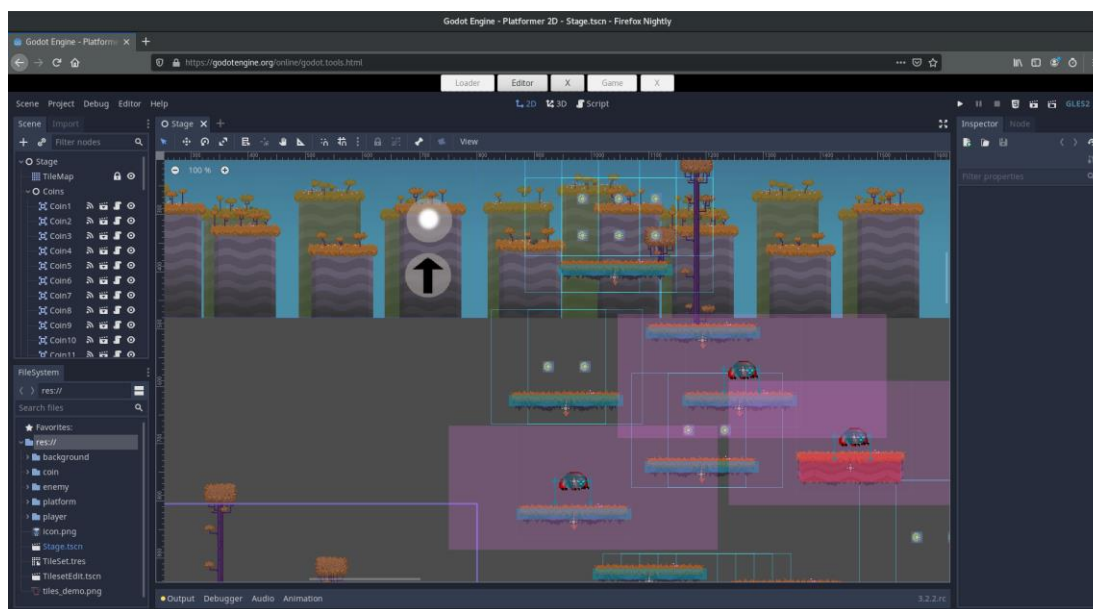
Двигателят поддържа широк набор от езици за програмиране, включително C++, C#, UnityScript и Boo.

Unity предлага и голям пазар, където разработчиците могат да изтеглят и купуват готови двуизмерни и триизмерни модели, текстури и анимации, които могат да използват в своите игри. [52, 53]

Множество известни игри като Pokemon Go, Angry Birds 2, Call of Duty, Cuphead, Monument Valley и много други са разработени на Unity. Но освен за игри се използва и в архитектурата, автомобилостроенето, киното, телевизията и в други индустрии. [54]

1.4.3 Godot

Godot е безплатен двигател с отворен код за създаване на двуизмерни и триизмерни игри, които могат да работят на множество платформи. Разработен е от Godot Engine през 2014 г. Предлага напълно интегрирана среда за разработка на видео игри. [55]



Фиг. 1.14 Интерфейс на Godot

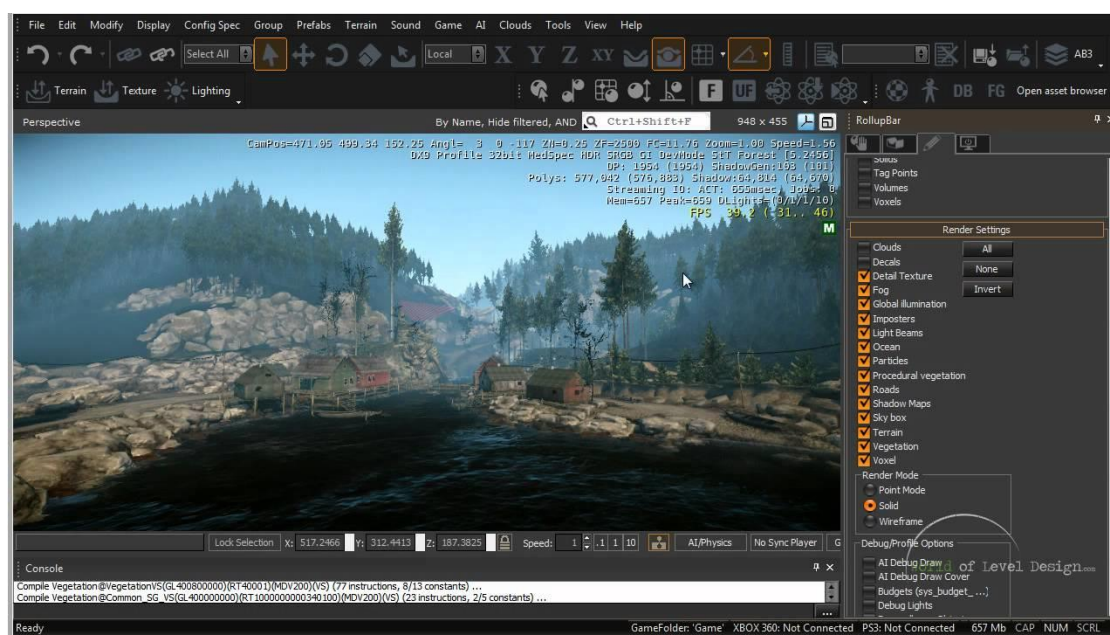
Една от ключовите характеристики на Godot е неговият фокус върху простотата и лекотата на използване, което го прави достъпна опция както за начинаещи, така и за опитни разработчици на игри.

Също така той предоставя визуален редактор и архитектура, базирана на възли, която позволява на програмистите да създават и редактират игрови елементи, герои и нива много лесно. [56]

Godot поддържа и широк набор от езици за програмиране – GDScript, C#, C и C++. Също предоставя и вградена система за физика и анимации. [57, 58, 59]

1.4.4 CryEngine

CryEngine е игрови двигател разработен от Crytek през 2002 г. Това е мощен двигател за игри, който е използван за създаването на много смайващи визуално игри като Crysis, Far Cry, Ryse: Son Of Rome и други. [60]



Фиг. 1.15 Интерфейс на CryEngine

Известен е със своите усъвършенствани графични възможности, глобално осветление в реално време и други функционалности, които му позволяват да създаде много детайлни и реалистични среди.

Двигателят поддържа езиките за програмиране C++, C# и Lua. [61]

1.4.5 Bevy

Bevy е двигател за игри с отворен код, управляван от данни, изграден върху Amethyst (Rust базиран двигател). Игровият двигател е изключително гъвкав. Проектиран е да бъде силно модуларен и адаптивен, което го прави лесен за използване в различни видове игри и приложения. Разполага с множество опции за персонализиране и модифициране. Използва ECS системата. [62, 63]

Една от ключовите характеристики на Bevy е неговата управлявана от данни архитектура, която позволява на разработчиците да дефинират и управляват игрови обекти и компоненти с помощта на прост, изразителен интерфейс за програмиране на приложения (Application Programming Interface). Това позволява лесно манипулиране на състоянието на играта и дава възможност за висока степен на гъвкавост в начина, по който се прилага логиката на играта. [62, 63]

Bevy също така включва вграден визуализатор и поддържа широка гама от графични и аудио технологии, включително DirectX 12, Vulkan и OpenAL. Освен това поддържа множество платформи, включително Windows, Linux и macOS. [62, 63]

Bevy е сравнително нов игрови двигател и все още е в процес на активно развитие. Използва се за създаване на различни типове игри и

приложения – от двуизмерни и триизмерни игри до преживявания с виртуална и разширена реалност.

1.5 Среди за създаване на видео игри

1.5.1 Visual Studio

Visual Studio е интегрирана среда за разработка (IDE), създадена от Microsoft. Visual Studio включва вграден редактор на код с функции като подчертаване на синтаксис, допълване на код и рефакторизиране (преструктуриране) на код. Той също така включва вграден „дебъгер“, който позволява на разработчиците да преминават през своя код и да намират и коригират грешки. Освен това има вграден терминал, интегриран контрол на версиите и поддръжка за широк набор от библиотеки. [64]

Visual Studio също така предоставя различни инструменти за изграждане и внедряване на приложения, включително поддръжка за изграждане на Windows Forms, WPF и UWP приложения, уеб разработка с ASP.NET и Azure и разработка на мобилни приложения с Xamarin, както и много други. [64]

1.5.2 CLion

CLion е междуплатформена интегрирана среда за разработка (IDE), създадена от JetBrains. CLion е проектиран да бъде лесен за използване, с модерен и интуитивен потребителски интерфейс и предоставя широка гама от функции, за да помогне на разработчиците да пишат код, да отстраняват грешки и да го преработват по-лесно. [65, 66]

CLion предлага и вграден терминал, както и поддръжка на широк набор от компилатори, включително GCC, Clang, MSVC, Cargo и други.

ВТОРА ГЛАВА

ФУНКЦИОНАЛНИ ИЗИСКВАНИЯ.

СТРУКТУРА НА ПРИЛОЖЕНИЕТО. ПОДБОР НА ТЕХНОЛОГИИ

2.1 Функционални изисквания към софтуерния продукт

Трябва да се създаде двуизмерна стратегическа игра за защитаване на база, реализирана посредством Rust и Bevy.

Функционалните изисквания са следните:

- Играчът започва с начален брой точки живот;
- Играчът започва с определен брой пари, с които може да закупи нови защити или да подобрява вече поставени върху полето постройки;
- Добавяне на поне 2 вида противници;
- Добавяне на поне 2 вида защитни кули;
- Генериране на вълни от противници. След натискането на бутон или след изтичането на някакъв таймер ще се генерират групи от противници, които да атакуват играча;
- Движение на враговете до базата. Противниците ще се движат автоматично по определена върху картата пътека. Когато стигнат края ѝ, те ще нанасят щета върху точките живот на играча в зависимост от това на колко кръв са стигнали до там. Тоест ако чудовище с 20 точки кръв стигне до базата, ще се отнемат 20 точки живот на играча;

- Функционалност за закупуване на защитни кули чрез графичен интерфейс – бутони;
- Възможност за поставяне на защитните кули върху самото поле. След натискането на бутон за закупуване на кула, играчът да може да постави селектираната кула някъде върху полето;
- Възможност за подобряване на статистиките на кулите. Всяка кула да има поне 3 ъпгрейда (подобрения);
- Кулите да имат няколко режима за приоритет на атака. Да могат да атакуват най-близкия до тях противник, както и най-далечния от тях, да могат да атакуват най-близкия до базата, тоест първия по пътеката, да могат да атакуват последния, тоест най-далечния от базата и също така да могат да атакуват най-силния и най-слабия, което ще определят по броя точки.

2.2 Избрани софтуерни средства

2.2.1 Игрови двигател

За игрови двигател бе избран Bevy. Макар и да е все още сравнително нов и да не е напреднал толкова, колкото други по-известни двигатели като Unity, Unreal и CryEngine, той все пак си има своите предимства. Едно от предимствата му пред други игрови двигатели, които ползват ООП (Обектно-ориентирано програмиране) като Unreal Engine, CryEngine и Godot, е че използва ECS (entity component system).



Фиг. 2.1 Игрови двигател Bevy

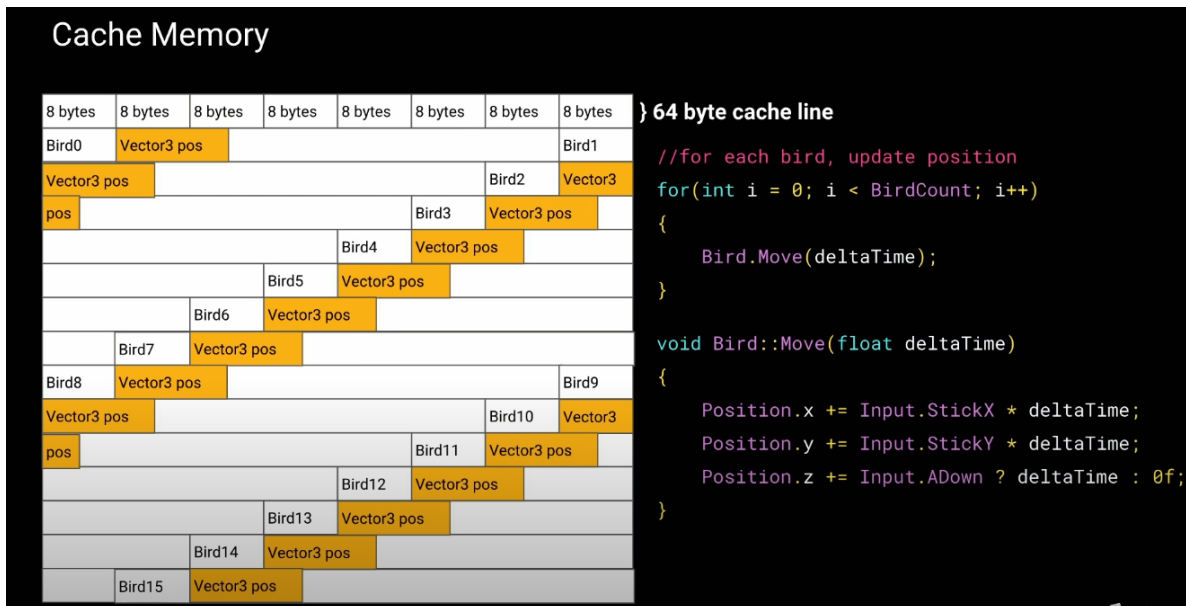
ECS представлява архитектурен модел, използван в разработката на игри и други приложения. ECS е начин за организиране на игрови обекти и техните компоненти, които съставляват един обект.

В ECS обектите (entities) са основните градивни елементи на играта, те представляват обектите в света на играта и нямат поведение или данни. В Bevy те са просто числа, или id-та. Компонентите са данните, които определят поведението и външния вид на даден обект. Те съдържат данни като позиция, точки живот, точки защита, точки атака и други. Компонентите в Bevy представляват структури, които съдържат няколко различни полета с данни. Системите са кодът, който изпълнява логиката на играта – те обработват обектите и техните компоненти, за да актуализират състоянието на играта. Системите в Bevy са имплементирани като функции, които приемат обектите и компонентите, от които се интересуват под формата на заявка и актуализират състоянието на играта според логиката вградена в тях.

Bevy ECS използва система за заявки, която позволява на разработчиците лесно да избират обектите и компонентите, от които се интересуват и искат да обработват. Системата за заявки се основава на „пакети“ и позволява на разработчиците да избират обекти въз основа на техните компоненти или да избират компоненти въз основа на обектите, към които са прикачени.

Едно от основните предимства на ECS пред ООП е, че подобрява производителността на играта и улеснява поддържането на кода. Чрез разделянето на данните и логиката става по-лесно идентифицирането и оптимизирането на някои по-бавни блокове код и системи. Освен това улеснява и добавянето на нови функции и тестването на играта. Обектно-ориентираното програмиране се фокусира върху обектите и тяхното поведение, докато ECS се фокусира върху разделянето на данните и логиката като отделни компоненти.

ECS подрежда данни от един и същи вид на едно и също място в паметта и се редуват данните на различните обекти. За разлика от ООП, което съхранява обектите с целите техни данни последователно – тоест съхранява обект след обект, ECS групира и съхранява данните от един и същи вид на едно място, което значително подобрява производителността и скоростта на кода (фиг. 2.2).



Фиг. 2.2 Запазване на данни в паметта чрез ECS системата

ECS набира популярност сред разработчиците на игри поради своите предимства в производителността. Улеснява разсъжденията за състоянието на играта и извършването на промени в нея. Той също така осигурява висока степен на гъвкавост в начина, по който се прилага логиката на играта, и улеснява добавянето на нови функции и тестването на играта.

2.2.2 Интегрирана среда за разработка

CLion бе избран за IDE, поради няколко различни причини:

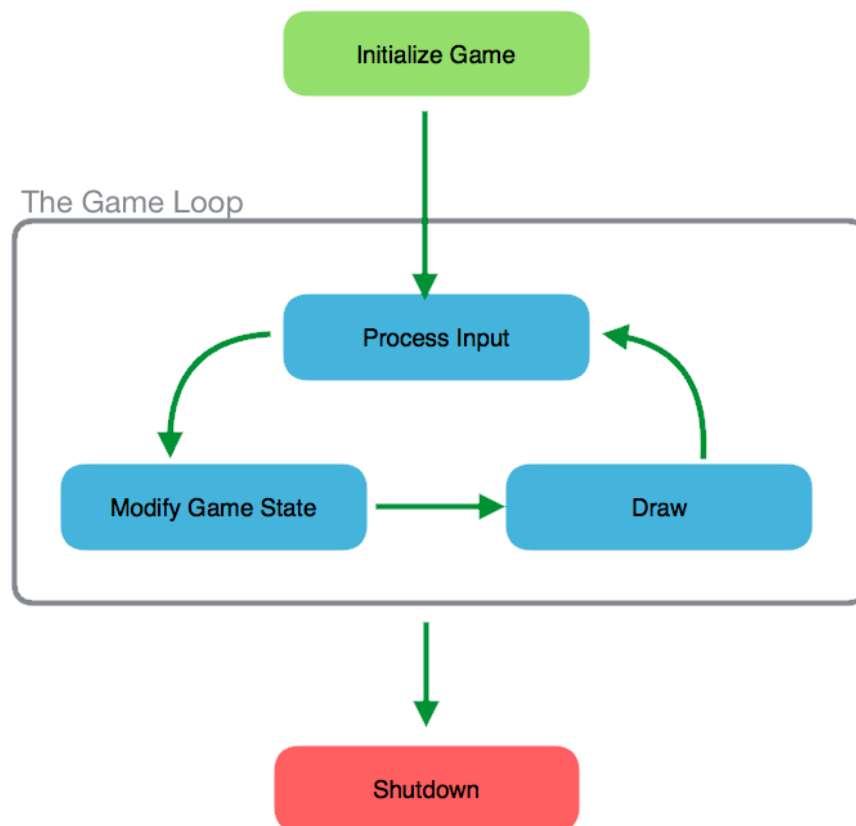
- Главно се използва за езици от ниско ниво като C, C++, Rust и други, и поддържа разработка с тях;
- Cross-platform – поддържа разработка на различни платформи – Windows, Linux, macOS;
- CLion е по-фокусиран върху етапа на разработка, докато Visual Studio има множество различни инструменти за изграждане и внедряване на всякакви приложения – уеб, мобилни и т.н.;

- Много удобно реструктуриране и форматиране на кода – CLion предлага огромен избор от опции за оптимизиране на код, пренаписване на код, преименуване на имена на функции, структури и променливи. Може да анализира контекста на кода и да предупреждава потребителя за грешки още преди компилация. Също така поддържа напреднала система за завършване на текста/кода, като спестява повтарящ се синтаксис и съответно време;
- Интеграция с Git. Позволява много лесно взаимодействие с Git и съответно GitHub;
- Опции за персонализиране на редактора – промяна на темата, големината на табулацията, цветовете, големината на шрифта и много други. Всичките тези опции могат да бъдат запазени върху акаунт, който да се използва на няколко различни устройства вместо да се преправят настройките на всяко устройство поотделно;

2.3 Алгоритми, използвани при разработването на видео игри

2.3.1 Цикъл на играта (Game loop)

Цикълът на играта е главна концепция в разработката на игри. Това е ядрото на логиката на играта и контролира хода на играта. Представлява повтарящ се процес, който актуализира състоянието на играта и изобразява визуалните елементи на играта. Цикълът на играта обикновено има два основни компонента: цикъл за актуализиране и цикъл за изобразяване (фиг. 2.3). [67]



Фиг. 2.3 Цикъл на играта

Цикълът за актуализиране е отговорен за обновяване на състоянието на играта – позицията на обектите, резултатите в играта и други. Обикновено се реализира като поредица от функции, които се извикват на фиксиран интервал от време, често 60 пъти в секунда. [67]

Цикълът за изобразяване е отговорен за показването на визуалните елементи на играта на екрана. Обикновено се реализира като функция, която се извиква след цикъла за актуализиране. Той използва актуализираното състояние на играта, за да визуализира обектите, фоновете и специалните ефекти на играта на екрана. [67]

Цикълът на играта също отговаря за обработката на потребителския вход, като въвеждане от клавиатура и мишка, и за управление на състоянието на играта, като пауза на играта или показване на менюто. [67]

Цикълът на играта е непрекъснат процес, който работи, докато играта не бъде изключена, и продължава да актуализира и изобразява играта, създавайки илюзията за движение и взаимодействие. Това е фундаментална концепция за разработване на игри и се прилага по различен начин в зависимост от използвания двигател на играта и езика за програмиране. [67]

2.3.2 Засичане на сблъсъци/колизии (Collision detection)

Алгоритмите за откриване на сблъсък се използват за определяне на това кога два обекта в играта са се сблъскали. Примери за алгоритми за откриване на сблъсък включват откриване на сблъсък в „ограничаваща“ кутия и откриване на сблъсък в „ограничаваща“ сфера. [68, 69]

Това е много важен алгоритъм в игрите от типа „защитна кула“, тъй като се използва за засичането на сблъсък между враговете и изстрелите от защитните кули, както и за движението на противниците по пътеката на картата.

2.3.3 Алгоритми за намиране на пътя (Pathfinding)

Алгоритмите за намиране на пътя се използват за изчисляване на пътя, по който враговете ще стигнат до базата на играча. Това е важно за играча, за да може стратегически да постави своите защитни кули, с които да блокира пътя на враговете. Най-често се използват A^* и алгоритъмът на Дейкстра за намиране на път.

2.3.4 Управление на вълни от противници

Алгоритмите за управление на вълните се използват за контролиране на генерирането на противници и тяхното поведение. Те могат да включват алгоритми, които контролират времето и броя на враговете, които се появяват във всяка вълна, както и типовете врагове, които се появяват.

2.3.5 Приоритет на атака

Алгоритми за насочване на атаката се използват, за да се определи към кой враг трябва да се насочи определената кула. Някои игри използват прости алгоритми, насочени към най-близкия до кулата враг, докато други използват по-усъвършенствани алгоритми, които вземат предвид фактори като точки живот на враговете, ефективността на кулите срещу различните видове врагове, разстоянието до базата и други.

2.3.6 Изкуствен интелект

Алгоритмите с изкуствен интелект се използват за контролиране на поведението на враговете. Те могат да включват алгоритми, които контролират движението на враговете и вземането на решения, като “flocking” и “decision tree”.

ТРЕТА ГЛАВА

ПРОГРАМНА РЕАЛИЗАЦИЯ НА ИГРА ОТ ТИП ЗАЩИТНА КУЛА НАПИСАНА С RUST И BEVY

3.1 Конструктор на приложението

Приложението се реализира с т.нар. „AppBuilder“. За да се създаде приложение, трябва да се дефинира структурата на проекта в тази функционалност на Bevy. Тук се добавят всички неща, които изграждат приложението – системи, събития, състояния, етапи, ресурси, плъгини и други.

Приложението (App) съхранява всички данни и всички системи.

```
fn main() {  
    App::new()  
        // Background of window. Set colour of screen on each refresh  
        .insert_resource(ClearColor(Color::rgb(0.1, 0.1, 0.1)))  
  
        // Add basic game functionality - window, game tick, renderer,  
        // asset loading, UI system, input, startup systems, etc.  
        .add_plugins(DefaultPlugins  
            // Prevent blurry sprites  
            .set(ImagePlugin::default_nearest())  
            .set(WindowPlugin {  
                window: WindowDescriptor {  
                    title: "Slimes Tower Defense".to_string(),  
                    position: WindowPosition::Centered,  
                    resizable: false,  
                    ..default()  
                },  
                ..default()  
            })  
        )  
}
```

Фиг. 3.1 Създаване на базово приложение

Създаваме ново приложение чрез `App::new()` . След това му добавяме най-основните функционалности като прозорец, визуализатор, „game tick“, зареждане на потребителския интерфейс чрез `.add_plugins(DefaultPlugins)` (фиг. 3.1).

Също така се добавя и ресурсът „ClearColor” чрез `.insert_resource(ClearColor(Color::rgb(0.1, 0.1, 0.1)))` . Това представлява глобален фонен цвят на визуализатора за изчистване на прозореца в началото на всеки кадър (фиг. 3.1).

За да се добави функционалността на приложението се използва командата `.add_plugin()` . „Плъгините“ групират свързани функционалности на едно място, за да е по-чист и организиран кодът.

```
// Plugins
.add_plugin(MainMenuPlugin)
.add_plugin(GameplayUIPlugin)
.add_plugin(MapPlugin)
.add_plugin(SettingsPlugin)
.add_plugin(AssetPlugin)
.add_plugin(PlayerPlugin)
.add_plugin(BasePlugin)
.add_plugin(TowerPlugin)
.add_plugin(TowerButtonPlugin)
.add_plugin(TowerSelectionPlugin)
.add_plugin(TowerUpgradePlugin)
.add_plugin(TowerUIPlugin)
.add_plugin(EnemyPlugin)
.add_plugin(WavePlugin)
.add_plugin(BulletPlugin)
.add_plugin(MovementPlugin)
```

Фиг. 3.2 Добавяне на функционалности на приложението

В конструкцията на приложението се добавя и началното състояние на играта. Състоянията (states) в bevy позволяват само на дадени системи да стартират по определено време. Първоначално състоянието на играта започва на главното меню и така ще се покаже само графичният интерфейс

на главното меню. Тоест в момента на стартиране на приложението не се стартират други функционалности като стрелянето на кулите (фиг. 3.3).

```
// Game State
.add_state(GameState::MainMenu)
```

Фиг. 3.3 Добавяне на начално състояние на играта – главното меню

3.2 Главно меню

Първоначално се създава главното меню чрез `MainMenuPlugin`. В него има 2 комплекта от системи. Първият се изпълнява само в началото на състоянието „MainMenu”. А вторият върви постоянно, докато програмата е в състояние „MainMenu” (фиг. 3.5).

```
pub struct MainMenuPlugin;

impl Plugin for MainMenuPlugin {
    fn build(&self, app: &mut App) {
        app
            .add_system_set(SystemSet::on_enter(GameState::MainMenu)
                .with_system(spawn_main_menu))
            .add_system_set(
                SystemSet::on_update(GameState::MainMenu)
                    .with_system(start_button_clicked)
                    .with_system(exit_button_clicked),
            );
    }
}
```

Фиг. 3.5 Добавяне на главното меню

В първия комплект е функцията `spawn_main_menu()`, която зарежда главното меню. В главното меню се зарежда заглавието на играта и 2 бутона – “Start Game” и „Exit Game”.

Във втория комплект от функции се реализира самата функционалност на 2-та бутон на главното меню. Бутонът „Start Game” служи за започването на играта. А бутонът „Exit Game” служи за изключване на приложението.

Когато се натисне бутонът за старт състоянието на играта се превключва от „MainMenu” към „Gameplay” (фиг. 3.6).

```
fn start_button_clicked(  
    mut commands: Commands,  
    interactions: Query<&Interaction, (With<StartButton>,  
    Changed<Interaction>)>,&br/>    menu_root: Query<Entity, With<MenuUIRoot>>,&br/>    mut game_state: ResMut<State<GameState>>,&br/>) {  
    for interaction in &interactions {  
        if matches!(interaction, Interaction::Clicked) {  
            let root_entity = menu_root.single();  
            commands.entity(root_entity).despawn_recursive();  
  
            game_state.set(GameState::Gameplay).unwrap();  
        }  
    }  
}
```

Фиг. 3.6 Функционалност на старт бутона

Когато се натисне бутонът за изход от играта, се извиква `EventWriter`, който съобщава събитие (Event) от вид `AppExit` и уведомява програмата да се изключи (фиг. 3.7).

Събитията в Bevy служат за изпращане на данни между различните системи. Позволяват комуникация между системите, чрез 2 структури – `EventReader<T>` и `EventWriter<T>`. Чрез `EventWriter` се изпращат събития, а чрез `EventReader` се получават и прочитат. Всяка система чете събитията самостоятелно, което позволява обработването на едни и същи събития от множество системи. [70]

```
fn exit_button_clicked(  
    interactions: Query<&Interaction, (With<ExitButton>,  
    Changed<Interaction>)>,&br/>    mut exit: EventWriter<AppExit>,&br/>) {  
    for interaction in &interactions {  
        if matches!(interaction, Interaction::Clicked) {  
            exit.send(AppExit);  
        }  
    }  
}
```

Фиг. 3.7 Функционалност на бутона за изход

3.3 Създаване на компонент за играч и компонент за база

3.3.1 Създаване на играч

Създава се компонент играч, чрез системата `spawn_player`, като му се задават първоначален брой пари. Чрез този компонент се контролира един от най-важните ресурси в играта – парите. Те служат за построяване на нови кули и за подобряване на техните статистики (фиг. 3.8).

```
#[derive(Component, Reflect, Default)]
#[reflect(Component)]
pub struct Player {
    pub money: usize,
}

fn spawn_player(mut commands: Commands) {
    commands.spawn((Player { money: 100 }, Name::new("Player")));
}
```

Фиг. 3.8 Създаване на играч

Играчът има 2 системи, които увеличават парите му. Първата `give_money_on_enemy_death()` чете събитията, свързани с противниците и когато някой противник е убит, системата увеличава парите на играча (фиг. 3.9).

```
fn give_money_on_enemy_death(
    mut player: Query<&mut Player>,
    mut death_events: EventReader<EnemyDeathEvent>,
) {
    let mut player = player.single_mut();
    for _ in death_events.iter() {
        player.money += 10;
    }
}
```

Фиг. 3.9 Възнаграждение на играча с пари за всеки убит противник

Функцията `give_money_on_wave_cleared` се изпълнява при получаване на събитието `WaveClearedEvent`. Тя обхожда всички обекти, които имат компонент от тип `Player` и увеличава паричния баланс на играча (фиг. 3.10).

```
fn give_money_on_wave_cleared(  
    mut player: Query<&mut Player>,  
    mut wave_events: EventReader<WaveClearedEvent>,  
) {  
    let mut player = player.single_mut();  
    for wave in wave_events.iter() {  
        player.money += wave.index + 101;  
    }  
}
```

Фиг. 3.10 Възнаграждение на играча с пари за всяка завършена вълна

Тези две системи изграждат логиката, която дава пари на играча, чрез които той може да закупува различни кули и да ги използва за защита срещу вълните от противници.

3.3.2 Създаване на база

Създаването на база става по подобен начин, както създаването на играча, като единствената разлика е, че на базата се добавя броя на точките живот вместо броя пари. Базата също е важен компонент, тъй като чрез нея се контролират точките живот на играча.

При достигане до последната точка от зададения им път, противниците нанасят щета на базата на играча. Това се реализира чрез функцията `damage_base()`. Първо се премахва противника от играта и след това функцията проверява колко точки живот остават на играча и се изваждат точките живот на противника от точките живот на играча. Ако кръвта на играча стане по-малка или равна на нула, играчът губи играта (фиг. 3.11).

```

pub fn damage_base(
    commands: &mut Commands,
    entity: &Entity,
    enemy_health: i32,
    base: &mut Base
) {
    commands.entity(*entity).despawn_recursive();

    if base.health > 0 {
        base.health -= enemy_health as i32;
    }
    else {
        base.health = 0;
        info!("GAME OVER");
    }
}

```

Фиг. 3.11 Функционалност за нанасяне на щета на базата на играча от противниците

3.4 Карта

3.4.1 Камера

Камерите са важен елемент в изработването на игри, тъй като те контролират гледната точка на играча и как той ще разглежда света на играта. Използват се за създаване на усещане на перспектива или дълбочина. В Bevy камерите определят изгледа в света на играта, като имат различни настройки като позиция, ориентация, зрително поле и други. Също така служат за изобразяване на потребителския интерфейс, който се показва над света на играта, точно пред камерата.

Кодът във фиг. 3.12 настройва двумерна камера за играта. Тя се центрира по средата на картата, като се изчислява централната позиция според ширината и височината на картата. Също така се настройва проекцията, така че да се мащабира изгледа на играта правилно, за да отговаря на размера на екрана.

```
fn setup_camera(mut commands: Commands, map: Res<Map>) {
    let mut camera = Camera2dBundle::default();
    camera.transform.translation.x = (map.width as f32 / 2. - 0.5) *
map.tile_size as f32;
    camera.transform.translation.y = (map.height as f32 / 2. - 0.5) *
map.tile_size as f32;
    camera.projection.scaling_mode = ScalingMode::Auto { min_width: 1280.,
min_height: 720.0 };
    commands.spawn((camera, MainCamera));
}
```

Фиг. 3.12 Настройване на камера

3.4.2 Зареждане на картата

Картата представлява решетка от плочки, т.нар. „tilemap“. Всяка плочка представлява различен вид терен. Може да е терен, на който могат да се поставят кули като трева или вода, или да е терен, по който се движат противниците.

Картата се зарежда от файл тип RON (Rusty Object Notation) (фиг. 3.13).

```
fn load_map(mut commands: Commands) {
    let f = File::open("./assets/data/map.ron").expect("Failed opening map file!");
    let mut map: Map = match ron::de::from_reader(f) {
        Ok(x) => x,
        Err(e) => {
            panic!("Failed to load map: {}", e);
        }
    };
};

let mut path_tiles = vec![];
let mut spawn: Point = Default::default();
let mut end: Point = Default::default();

map.tiles.reverse();

for (y, row) in map.tiles.iter().enumerate() {
    for (x, tile) in row.iter().enumerate() {
        match tile {
            Tile::Spawn => spawn = Point {x, y},
            Tile::Path(_) => path_tiles.push(Point {x, y}),
            Tile::End => end = Point {x, y},
            _ => {}
        }
    }
}

map.checkpoints.push(spawn.to_vec3());
map.create_checkpoints(path_tiles, spawn, end);
commands.insert_resource(map);
}
```

Фиг. 3.13 Зареждане на картата от RON файл

Първоначално се прочита файлът и се проверяват всичките плочки, за да се намерят всички плочки с тип „Path”, които маркират пътеката на противниците. След като се запишат се извиква функцията `create_checkpoints()`, която нарежда плочките на пътеката в списък (фиг. 3.14).

```
fn create_checkpoints(&mut self, mut path_tiles: Vec<Point>, spawn: Point,
end: Point) {
    let mut last_point = spawn;

    loop {
        let next_point_position = path_tiles.iter().position(|point| {
            let length = Vec2::new(
                last_point.x as f32 - point.x as f32,
                last_point.y as f32 - point.y as f32,
            ).length();
            length >= 0.9 && length <= 1.1
        });
        if let Some(next_point_position) = next_point_position {
            let next_point = path_tiles.remove(next_point_position);
            self.checkpoints.push(Coordinate {
                x: (next_point.x * self.tile_size) as f32,
                y: (next_point.y * self.tile_size + self.tile_size / 2) as f32,
            }.to_vec3());
            last_point = next_point;
        }
        else {
            self.checkpoints.push(Coordinate {
                x: (end.x * self.tile_size) as f32,
                y: (end.y * self.tile_size + self.tile_size / 2) as f32,
            }.to_vec3());
            self.checkpoints[0] = Coordinate {
                x: self.checkpoints[0].x * self.tile_size as f32,
                y: self.checkpoints[0].y * self.tile_size as f32,
            }.to_vec3();

            return;
        }
    }
}
```

Фиг. 3.14 Нареждане на плочките на пътеката на противниците

3.4.3 Актуализация на пътя на противниците

Картата представлява ресурс и съдържа големината на картата, двумерен списък от всичките плочки, големината на една плочка и

контролните точки на пътеката, през които трябва да преминат противниците (фиг. 3.15).

```
#[derive(Resource, Serialize, Deserialize)]
pub struct Map {
    pub width: usize,
    pub height: usize,
    pub tiles: Vec<Vec<Tile>>,
    pub tile_size: usize,
    pub checkpoints: Vec<Vec3>
}
```

Фиг. 3.15 Структура за картата

Движението на противниците по пътеката се реализира от две системи – `update_enemy_checkpoint()` и `despawn_enemy()`.

```
fn update_enemy_checkpoint(
    mut enemies: Query<(&mut Movement, &mut Transform, &mut Path)>,
    map: Res<Map>,
    time: Res<Time>
) {
    for (mut movement,
        mut transform,
        mut path) in &mut enemies {
        if path.index >= map.checkpoints.len() {
            continue;
        }

        let distance = map.checkpoints[path.index] - transform.translation;
        if distance == Vec3::ZERO {
            path.index += 1;
            continue;
        }

        let enemy_movement = distance.normalize() * movement.speed *
            time.delta_seconds();

        if enemy_movement.length() > distance.length() {
            transform.translation = map.checkpoints[path.index];
            movement.distance_travelled += distance.length();
            movement.direction = map.checkpoints[path.index] -
transform.translation;
            path.index += 1;
        }
        else {
            movement.distance_travelled += enemy_movement.length();
            movement.direction = map.checkpoints[path.index] -
transform.translation;
            transform.translation += enemy_movement;
        }
    }
}
```

Фиг. 3.16 Актуализиране на пътя на противниците

Пътят на противниците се актуализира по предварително определен път представен от поредица от плочки. Функцията преминава през всички противници и проверява дали са достигнали последната точка от картата. Ако не са достигнали тази точка, функцията изчислява разстоянието между следващата точка от пътеката и сегашната позиция на противника. Ако разстоянието е нула, врагът е достигнал контролната точка и се увеличава достигнатият от пътеката индекс и противникът започва да се движи към следващата контролна точка. Ако разстоянието не е нула, функцията изчислява разстоянието, което ще измине противникът, и ако то е по-голямо от разстоянието между него и дадената точка от картата, врагът се премества до позицията на следващата точка. В противен случай врагът се придвижва към следващата точка с изчисленото разстояние.

Другата система, която участва в движението на противниците (фиг. 3.17), проверява до коя точка от пътеката са достигнали враговете и ако са преминали последната точка се изпълнява функцията `damage_base()` описана във фиг. 3.11.

```
fn despawn_enemy(  
    mut commands: Commands,  
    mut enemies: Query<(Entity, &Enemy, &mut Path)>,  
    mut base: Query<&mut Base>,  
    map: Res<Map>  
) {  
    let mut base = base.single_mut();  
  
    for (entity,  
        enemy,  
        path) in &mut enemies {  
        if path.index >= map.checkpoints.len() {  
            damage_base(&mut commands, &entity, enemy.health, &mut base);  
        }  
    }  
}
```

Фиг. 3.17 Нанасяне на щета върху базата от противниците

3.5 Противници

Противниците в играта се движат по дадена пътека и целта им е да стигнат до края ѝ, за да нанесат щета върху точките живот на играча. Те притежават свои собствени точки живот и когато стигнат края на пътеката, те ще нанесат по-голяма щета върху базата, ако имат повече кръв. Целта на играча е да им попречи да достигнат базата му (фиг. 3.18).

```
#[derive(Reflect, Component, Default, Clone, Serialize, Deserialize)]
#[reflect(Component)]
pub struct Enemy {
    pub health: i32
}
```

Фиг. 3.18 Компонент за противници

За да се реализират всички функционалности на противниците, са нужни няколко различни компоненти, които се добавят върху един обект. Всичките компоненти се добавят наведнъж чрез `EnemyBundle` структурата (фиг. 3.22), която представлява пакет от компоненти. Пакетите (Bundles) в Bevy са начин за обединяване на свързани помежду си компоненти, за да се създаде общ обект. Те улесняват кода и спестяват повтаряне на множество редове код. [71] `EnemyBundle` обединява:

- компонента `Enemy`, който описва самия противник (фиг. 3.18)
- компонента `EnemyType`, който описва типа на противника и съответно колко е силен (фиг. 3.19)

```
#[derive(Component, Display, Clone, Copy, Debug, PartialEq, Eq, Deserialize,
Serialize, Hash)]
pub enum EnemyType {
    Green,
    Yellow,
    Pink,
    White,
    Blue,
    Orange,
    Purple,
    Red
}
```

Фиг. 3.19 Компонент за типа на противника

- КОМПОНЕНТИТЕ `Movement` и `Path`, КОИТО СЕ ИЗПОЛЗВАТ ЗА ДВИЖЕНИЕТО НА ПРОТИВНИЦИТЕ ПО ПЪТЕКАТА. В `Movement` се съхранява информация за посоката, в която се движи противника, разстоянието, което е изминал и неговата скорост. В `Path` се съхранява поредната точка от пътеката, която е успял да достигне противника (фиг. 3.20)

```
#[derive(Reflect, Component, Clone, Default, Serialize, Deserialize)]
#[reflect(Component)]
pub struct Movement {
    pub direction: Vec3,
    pub speed: f32,
    pub distance_travelled: f32
}

#[derive(Reflect, Component, Default, Clone, Serialize, Deserialize)]
#[reflect(Component)]
pub struct Path {
    pub index: usize
}
```

Фиг. 3.20 Компоненти за движение на противниците по пътеката

- КОМПОНЕНТИ ЗА АНИМАЦИЯ НА МОДЕЛА НА ПРОТИВНИКА — `AnimationIndices` и `AnimationTimer` (3.21)

```
#[derive(Component, Deref, DerefMut, Serialize, Deserialize, Clone)]
pub struct AnimationTimer(pub Timer);

#[derive(Component, Serialize, Deserialize, Clone)]
pub struct AnimationIndices {
    pub first: usize,
    pub last: usize
}
```

Фиг. 3.21 Компоненти за анимация на модела на противника

```
#[derive(Bundle, Serialize, Deserialize, Clone)]
pub struct EnemyBundle {
    pub enemy_type: EnemyType,
    pub enemy: Enemy,
    pub movement: Movement,
    pub animation_indices: AnimationIndices,
    pub animation_timer: AnimationTimer,
    pub path: Path,
    pub name: Name
}
```

Фиг. 3.22 Пакет от компоненти, изграждащи противниците

3.5.1 Компонента за противници

Противниците, освен да се движат, имат система, която проверява кръвта им, и ако е по-малка или равна на нула, ги премахват от играта, защото те умират. Когато умират се изпраща събитие в играта, което се чете от системата на играча `give_money_on_enemy_death()`, описана във фиг. 3.9 и тя дава пари на играча за победения враг (фиг. 3.23).

```
fn despawn_enemy_on_death(  
    mut commands: Commands,  
    enemies: Query<(Entity, &mut Enemy)>,  
    mut death_event_writer: EventWriter<EnemyDeathEvent>  
) {  
    for (entity, enemy) in &enemies {  
        if enemy.health <= 0 {  
            death_event_writer.send(EnemyDeathEvent);  
            commands.entity(entity).despawn_recursive();  
        }  
    }  
}
```

Фиг. 3.23 Премахване на противниците от играта

3.5.2 Типове противници

Различните типове противници се зареждат от файл в ресурса `EnemyTypeStats` по същия начин както картата (фиг. 3.24).

```
pub fn load_enemy_type_stats(  
    mut commands: Commands  
) {  
    let f = File::open("./assets/data/enemy_stats.ron")  
        .expect("Failed opening enemy stats file!");  
    let enemy_type_stats: EnemyTypeStats = match ron::de::from_reader(f) {  
        Ok(x) => x,  
        Err(e) => {  
            panic!("Failed to load enemy stats: {}", e);  
        }  
    };  
    commands.insert_resource(enemy_type_stats);  
}
```

Фиг. 3.24 Зареждане на типовете противници от файл

3.5.3 Анимация на модела на противниците

Противниците представляват „слаймове“, които подскочат докато се движат. Скачането се осъществява чрез зареждане на т.нар. „Sprite Sheet“. Тези листа от изображения съдържат множество малки модели, които се използват за анимация и графика във видео игри. Отделните изображения, наречени „спрайтове“ най-често се подреждат в решетка и могат индивидуално да се достъпват чрез указване на позицията и размера им в листа. Чрез тях се намалява броят на файловете с изображения и анимацията се реализира по-бързо. В Bevy те се осъществяват чрез `TextureAtlasSprite` (фиг. 3.25).

```
fn animate_enemy_sprite(  
    time: Res<Time>,  
    mut query: Query(<  
        &AnimationIndices,  
        &mut AnimationTimer,  
        &mut TextureAtlasSprite,  
        &Movement  
    >)  
) {  
    for (indices,  
        mut timer,  
        mut sprite,  
        movement) in &mut query {  
        // Change direction based on where enemy is heading  
        if movement.direction.x != 0. {  
            if movement.direction.x < 0. {  
                sprite.flip_x = true;  
            } else {  
                sprite.flip_x = false;  
            }  
        }  
        // Animate sprite  
        timer.tick(time.delta());  
  
        if timer.just_finished() {  
            if sprite.index == indices.last {  
                sprite.index = indices.first;  
            }  
            else {  
                sprite.index += 1;  
            }  
        }  
    }  
}
```

Фиг. 3.25 Анимация на „спрайт шийт“

Първоначално се обхождат всички обекти с компонентите `AnimationIndices` и `AnimationTimer`, тоест всички противници. След това се проверява накъде се насочват и според това се обръща модела им да гледа наляво или надясно и това се осъществява като се обърне x-координатът на модела. След това на всяко изтичане на таймера се увеличава индексът на модела, за да се вземе следващият анимационен модел, или ако е последният модел от анимацията, се взима първият (фиг. 3.25).

3.5.4 Вълни от противници

Противниците се групират на вълни, за да образуват по-силна атака към играча. Вълната представлява предварително дефинирана последователност от врагове, срещу които играчите трябва да се защитават на всяко ниво на играта. Вълните могат да варират по брой противници, сила на противниците, паричната награда на края на всяка вълна и прогресивно стават все по-трудни с напредването на играча през играта.

Алгоритъмът за управление на вълни определя трудността на играта. Той управлява динамичността на играта и провокира играча да адаптира стратегиите си към вълните и да поставя нужните кули, които са най-ефективни срещу дадените противници. Различните вълни награждават играча с различна сума от пари, според трудността им и по този начин лимитират възможността на играча да поставя прекалено силни кули за дадена вълна от противници. Така се регулира икономиката в играта, за да се предостави предизвикателно преживяване на играча и да не е прекалено трудна играта.

Управлението на вълните се осъществява чрез 3 структури – компонента `Wave`, и ресурсите `Waves` и `WaveState`.

В компонента `Wave` в `enemies` се съхраняват всички противници и след какво време ще тръгне следващият противник, а в `current` се съхранява индексът на сегашния противник (фиг. 3.26).

```
#[derive(Component, Deserialize)]
pub struct Wave {
    pub enemies: Vec<(EnemyType, Duration)>,
    pub current: usize // Current enemy
}
```

Фиг. 3.26 Компонент за вълна

В ресурса `Waves` е подобна логиката, като там се съхраняват всички вълни противници и индексът на настоящата вълна (фиг. 3.27).

```
#[derive(Resource, Default, Deserialize)]
pub struct Waves {
    pub waves: Vec<Wave>,
    pub current: usize
}
```

Фиг. 3.27 Компонент за всички вълни

В ресурса `WaveState` има 3 компонента, които се използват за проследяването на състоянието на настоящата вълна. Чрез `wave_spawn_timer` се отчита колко време трябва да измине преди започването на следващата вълна. Чрез `enemy_spawn_timer` се отчита изминатото време между появяването на всеки от противниците. Този таймер се използва за да не се пускат всички противници едновременно и да изглежда, че се движи само един противник, но в действителност да са се натрупали няколко противника в една и съща позиция. И чрез `remaining` се изчислява колко противници не са се появили все още от настоящата вълна (фиг. 3.28).

```
#[derive(Resource)]
pub struct WaveState {
    pub wave_spawn_timer: Timer,
    pub enemy_spawn_timer: Timer,
    pub remaining: usize
}
```

Фиг. 3.28 Ресурс за управляване на състоянието на настоящата вълна

Тези 3 структури си имат и собствени методи, които изпълняват определени функции.

Ресурсът за вълни `Waves` има 2 метода – `current()` и `advance()`. Методи се добавят на структурите в Rust чрез т.нар. „блокове за изпълнение“ (implementation blocks). „Impl“ блокът е блок от код, който осигурява имплементацията на някаква функционалност за дадена структура или „enum“. Всичко в този „impl“ блок ще бъде свързано с типа структура, към която е имплементирано. Първият метод – `current()`, връща настоящата вълна, а вторият преминава към следващата вълна като изпраща събитие за завършена вълна от тип `WaveClearedEvent` и увеличава индекса на вълните и връща новата вълна (фиг. 3.29).

```
impl Waves {
    pub fn current(&self) -> Option<&Wave> {
        return self.waves.get(self.current);
    }

    pub fn advance(
        &mut self,
        wave_cleared_writer: &mut EventWriter<WaveClearedEvent>
    ) -> Option<&Wave> {
        wave_cleared_writer.send(WaveClearedEvent {index: self.current});
        self.current += 1;
        return self.current();
    }
}
```

Фиг. 3.29 Методи на структурата за вълните от чудовища

Първоначално се зареждат всичките вълни от файл и се имплементират като ресурс от тип `Waves` и се създава и ресурс от тип `WaveState` (фиг. 3.30).

```
fn load_waves(
    mut commands: Commands
) {
    let f = File::open("./assets/data/waves.ron").expect("Failed opening wave file!");
    let waves: Waves = match ron::de::from_reader(f) {
        Ok(x) => x,
        Err(e) => {
            panic!("Failed to load waves: {}", e);
        }
    };

    let num_enemies = waves.waves[0].enemies.len();

    commands.insert_resource(waves);
    commands.insert_resource(WaveState {
        wave_spawn_timer: Timer::new(Duration::from_secs(10), TimerMode::Once),
        enemy_spawn_timer: Timer::new(Duration::from_millis(1),
                                       TimerMode::Repeating),
        remaining: num_enemies
    })
}
```

Фиг. 3.30 Зареждане на вълните от противници от файл и създаване на ресурси за имплементацията на вълните

Системата, която управлява вълните е `spawn_waves()`. Първоначално функцията проверява дали всички врагове в текущата вълна са излезли на полето. Ако са излезли, тогава таймерът започва да отброява до появата на следващата вълна. Когато таймерът приключи се взема следващата вълна, ако съществува и се актуализира състоянието на настоящата вълна (фиг. 3.31).

```
if wave_state.remaining == 0 {
    wave_state.wave_spawn_timer.tick(time.delta());
    if !wave_state.wave_spawn_timer.just_finished() {
        return;
    }
    if let Some(next_wave) = waves.advance(&mut wave_cleared_writer) {
        wave_state.remaining = next_wave.enemies.len();
        commands.insert_resource(WaveState::from((next_wave,
next_wave.enemies.len())));
    }
}
```

Фиг. 3.31 Проверка за преминаване към следващата вълна от чудовища

След това се отброява таймерът за раждане на следващия противник. Когато той изтече, се изчислява индексът на противника, който трябва да бъде пуснат от системата, като се вади броят на останалите противници от вълната от общия брой на противниците в настоящата вълна. И след това се ражда нов противник от дадения тип върху картата на първата начална точка на пътеката, като на функцията за раждане на нов противник се подават като аргументи координатите на първата позиция от картата, пътеката, статистиките на вида противник и графиките на играта (фиг. 3.32).

```
let Some(current_wave) = waves.current() else {
    return;
};

wave_state.enemy_spawn_timer.tick(time.delta());
if !wave_state.enemy_spawn_timer.just_finished() {
    return;
}

let index = current_wave.enemies.len() - wave_state.remaining;

spawn_enemy(&mut commands,
            &map_path,
            current_wave.enemies[index].0,
            &assets,
            map_path.checkpoints[0],
            Path { index: 0 },
            &enemy_stats);
```

Фиг. 3.32 Раждане на противник от вълната

След това се актуализира таймерът за раждане на следващия противник и се обновява състоянието на вълната като се премахва 1 от брояча на оставащите противници във вълната (фиг. 3.33).

```
wave_state.enemy_spawn_timer = Timer::new(
    current_wave.enemies[index].1,
    TimerMode::Repeating);

wave_state.remaining -= 1;
```

Фиг. 3.33 Актуализация на състоянието на вълната

3.6 Защитни кули

3.6.1 Поставяне на защитни кули

Първоначално се прочита потребителският вход, след това се проверява дали играчът има достатъчно пари, за да закупи нова защитна кула и след това се създава модел на кулата, който следва мишката докато не бъде поставена кулата. Създаването на защитни кули може да се осъществи по 2 начина:

– чрез клавиши от клавиатурата (фиг. 3.34)

```
// Keyboard shortcuts
if query.is_empty() { // Spawn one tower at a time
    if keys.just_pressed(KeyCode::Key1) &&
        player.money >= TowerType::Nature.get_price() as usize {
        info!("Spawning: Nature wizard");

        spawn_sprite_follower(&mut commands, window, camera, camera_transform,
                               &mut meshes, &mut materials, &TowerType::Nature,
                               &assets);
    }
    else if keys.just_pressed(KeyCode::Key2) &&
        player.money >= TowerType::Fire.get_price() as usize {
        info!("Spawning: Fire wizard");

        spawn_sprite_follower(&mut commands, window, camera, camera_transform,
                               &mut meshes, &mut materials, &TowerType::Fire,
                               &assets);
    }
    else if keys.just_pressed(KeyCode::Key3) &&
        player.money >= TowerType::Ice.get_price() as usize {
        info!("Spawning: Ice wizard");

        spawn_sprite_follower(&mut commands, window, camera, camera_transform,
                               &mut meshes, &mut materials, &TowerType::Ice,
                               &assets);
    }
    else if keys.just_pressed(KeyCode::Key4) &&
        player.money >= TowerType::Dark.get_price() as usize {
        info!("Spawning: Dark wizard");

        spawn_sprite_follower(&mut commands, window, camera, camera_transform,
                               &mut meshes, &mut materials, &TowerType::Dark,
                               &assets);
    }
    else if keys.just_pressed(KeyCode::Key5) &&
        player.money >= TowerType::Mage.get_price() as usize {
        info!("Spawning: Mage wizard");

        spawn_sprite_follower(&mut commands, window, camera, camera_transform,
                               &mut meshes, &mut materials, &TowerType::Mage,
                               &assets);
    }
    else if keys.just_pressed(KeyCode::Key6) &&
        player.money >= TowerType::Archmage.get_price() as usize {
        info!("Spawning: Archmage wizard");

        spawn_sprite_follower(&mut commands, window, camera, camera_transform,
                               &mut meshes, &mut materials, &TowerType::Archmage,
                               &assets);
    }
}
```

Фиг. 3.34 Създаване на кула чрез потребителски вход от клавиатурата

– чрез бутони от графичния интерфейс (фиг. 3.35)

```
for (interaction, tower_type, state) in &interaction {
    if player.money >= state.price as usize {
        match interaction {
            Interaction::Clicked => {
                info!("Spawning: {tower_type} wizard");
                if query.is_empty() { // Spawn one tower at a time
                    // Change button UI
                    for (mut image, button_tower_type) in images.iter_mut() {
                        if button_tower_type == tower_type {
                            image.0 = assets.get_button_pressed_asset(*tower_type);
                        }
                    }

                    // Spawn tower sprite following mouse
                    spawn_sprite_follower(&mut commands, window, camera,
                                         camera_transform,
                                         &mut meshes, &mut materials, tower_type,
                                         &assets);
                }
            }
            Interaction::Hovered => {
                // Change button UI
                for (mut image, button_tower_type) in images.iter_mut() {
                    if button_tower_type == tower_type {
                        image.0 = assets.get_button_hovered_asset(*tower_type);
                    }
                }
            }
            Interaction::None => {
                // Change button UI
                for (mut image, button_tower_type) in images.iter_mut() {
                    if button_tower_type == tower_type {
                        image.0 = assets.get_button_asset(*tower_type);
                    }
                }
            }
        }
    }
}
```

Фиг. 3.35 Създаване на кула чрез потребителски вход от бутони на графичния интерфейс

След като е прочетен входът на потребителя се създава модел на избрания тип кула, който да следва мишката. Това се реализира чрез функцията `spawn_sprite_follower()` (фиг. 3.36).

```

fn spawn_sprite_follower(
    commands: &mut Commands,
    window: &Window,
    camera: &Camera,
    camera_transform: &GlobalTransform,
    meshes: &mut ResMut<Assets<Mesh>>,
    materials: &mut ResMut<Assets<ColorMaterial>>,
    tower_type: &TowerType,
    assets: &Res<GameAssets>
) {
    // Spawn component that alerts the place_tower() system that a button has
    // been pressed
    // and it starts moving a sprite with the cursor until the tower is placed
    if let Some(position) = window.cursor_position() {
        let transform = window_to_world_pos(window, position, camera,
        camera_transform);
        commands.spawn(SpriteBundle {
            texture: assets.get_tower_asset(*tower_type),
            transform: Transform::from_translation(transform),
            ..default()
        })
        .insert(MaterialMesh2dBundle {
            mesh: meshes.add(shape::Circle::new(tower_type.get_range() as
f32).into()).into(),
            material: materials.add(ColorMaterial::from(
                Color::rgba_u8(0, 0, 0, 85))),
            transform: Transform::from_translation(Vec3::new(transform.x,
transform.y, 0.)),
            ..default()
        })
        .insert(SpriteFollower)
        .insert(*tower_type);
    }
}

```

*Фиг. 3.36 Създаване на модел, който да следва мишката,
докато не бъде поставен*

Моделът следва мишката в системата, като си променя цвета на кръга около него (показващ атакуващия обхват на кулата), според това дали мишката се намира на валидни за поставяне на кула координати. Например ако мишката се намира върху друга вече поставена кула, играчът няма да може да постави селектираната кула върху нея и обхватът на селектираната кула ще стане червен, докато мишката не се премести от невалидните координати (фиг. 3.37).

```

for (entity, mut transform, tower_type, mut color) in query.iter_mut() {
    if !clicked_tower.is_empty() {
        let entity = clicked_tower.single_mut();
        commands.entity(entity).remove::<(Handle<ColorMaterial>,
TowerUpgradeUI)>();
    }
    // Sprite follows mouse until tower is placed or discarded
    if let Some(position) = window.cursor_position() {
        transform.translation =
            window_to_world_pos(window, position, camera, camera_transform);
        for tower_transform in towers.iter() {
            // Tower range when trying to place on path/invalid tile
            if Vec3::distance(transform.translation, tower_transform.translation)
<= 40. {
                *color = materials.add(ColorMaterial::from(
                    Color::rgba_u8(202, 0, 0, 150)));
            }
            else {
                *color = materials.add(ColorMaterial::from(
                    Color::rgba_u8(0, 0, 0, 85)));
            }
        }
    }
}
}

```

Фиг. 3.37 Модел на селектираната кула следва мишката

Моделът може да спре да следва мишката по два начина. Първият е когато потребителят натисне левия бутон на мишката, за да постави избраната от него защитна кула. Ако мишката се намира на валидна позиция, кулата се поставя върху картата. В противен случай не се поставя и моделът продължава да следва мишката. Вторият начин е ако се натисне десния бутон на мишката или тя излезе от екрана, за да се откаже играчът от поставянето на кулата (фиг. 3.38).

Преди да се постави кулата се вади цената на кулата от парите на играча и се премахва моделът, който следва мишката.

След това кулата се поставя върху полето с функцията `spawn_tower()` (фиг. 3.39). Тази функция приема типа на избраната кула и координатите на мишката и създава обект от тип `Tower`, като взема статистиките на избрания тип кула чрез функцията `get_tower()` и поставя

кулата на координатите на мишката. Също така се добавя и компонента `TowerUpgradeUI`, така че автоматично да бъде селектирана новопоставената кула.

```
fn place_tower {
    for (entity, mut transform, tower_type, mut color) in query.iter_mut() {
        // Spawn the tower if user clicks with mouse button in a valid tower
        placement zone
        if mouse.just_pressed(MouseButton::Left) {
            if let Some(screen_pos) = window.cursor_position() {
                let mouse_click_pos =
                    window_to_world_pos(window, screen_pos, camera, camera_transform);

                let mut place_tower = true;

                for tower_transform in towers.iter() {
                    if Vec3::distance(mouse_click_pos, tower_transform.translation) <=
40. {
                        place_tower = false;
                    }
                }
                if place_tower {
                    player.money -= tower_type.get_price() as usize;
                    commands.entity(entity).despawn_recursive();
                    spawn_tower(&mut commands,
                                *tower_type,
                                &assets,
                                mouse_click_pos, &mut meshes, &mut materials);
                }
            } // Discard tower
            else if mouse.just_pressed(MouseButton::Right) ||
window.cursor_position().is_none() {
                commands.entity(entity).despawn_recursive();
            }
        }
    }
}
```

Фиг. 3.38 Функционалност за поставяне на кула на валидно място

```
pub fn spawn_tower(
) {
    commands.spawn(tower_type.get_tower(assets, position))
        .insert(MaterialMesh2dBundle {
            mesh: meshes.add(shape::Circle::new(tower_type.get_range() as
f32).into())
                .into(),
            material: materials.add(ColorMaterial::from(
                Color::rgba_u8(0, 0, 0, 85))),
            transform: Transform::from translation(Vec3::new(
                position.x, position.y, 0.)),
            ..default()
        }).insert(TowerUpgradeUI);
}
```

Фиг. 3.39 Функционалност за създаване на кула

3.6.2 Стреляне на защитните кули

Алгоритъмът за стреляне на защитните кули започва като за всяка кула на полето проверява дали има противник в нейния обхват. Ако има му намира посоката, според приоритета на насочване на атаката кулата. Ако няма противник се нулира таймера за стрелба и се променя вътрешен за кулата флаг `first_enemy_appeared`. Когато се появи първият противник кулата трябва веднага да започне да стреля без да чака таймера и за това се използва флага (фиг. 3.40).

```
fn tower_shooting() {
    for (tower_entity,
        mut tower,
        tower_type,
        mut tower_transform,
        transform) in &mut towers {
        // Check if an enemy is in range so we can tick the timer
        if enemy_in_range(&tower, &tower_transform, &enemies) {
            let bullet_spawn_pos = transform.translation() +
tower.bullet_spawn_offset;

            let direction = match &tower.target {
                FIRST => first_enemy_direction(&enemies,
                                                bullet_spawn_pos,
                                                tower.range),
                LAST => last_enemy_direction(&enemies,
                                             bullet_spawn_pos,
                                             tower.range),
                CLOSE => closest_enemy_direction(&enemies,
                                                  bullet_spawn_pos,
                                                  tower.range),
                STRONG => strongest_enemy_direction(&enemies,
                                                    bullet_spawn_pos,
                                                    tower.range),
                WEAK => weakest_enemy_direction(&enemies,
                                                bullet_spawn_pos,
                                                tower.range)
            };
        }
        else {
            tower.shooting_timer.reset();
            tower.first_enemy_appeared = true;
        }
    }
}
```

Фиг. 3.40 Проверка дали има противник в обхвата на кулата и намиране на неговата посока

Ако е намерена посоката на някой противник, кулата проверява дали е свършил таймера за стрелба. Ако е свършил, кулата изчислява ъгъла между себе си и противника и според ъгъла върти модела си, за да гледа в посоката му преди да стреля. След това се създава нов обект от тип `Bullet`, който се насочва в посоката на противника (фиг. 3.41).

```
// If there is an enemy in the tower's range (if direction != None), then
shoot bullet
if let Some(direction) = direction {
    // If the attack cooldown finished OR if there was no enemy spawned before,
    spawn bullet
    if tower.shooting_timer.just_finished() || tower.first_enemy_appeared {
        tower.first_enemy_appeared = false;

        // Calculate angle between tower and enemy
        let mut angle = direction.angle_between(tower.bullet_spawn_offset);
        if tower.bullet_spawn_offset.y > direction.y { // flip angle if enemy is
below tower
            angle = -angle;
        }

        // Rotate tower to face enemy it is attacking, based on enemy's location
        tower_transform.rotation = Quat::from_rotation_z(angle);

        // Make bullet a child of tower
        commands.entity(tower_entity).with_children(|commands| {
            commands.spawn(tower_type.get_bullet(
                tower.damage,
                &assets,
                Transform::from_translation(tower.bullet_spawn_offset)));
        });

        tower.shooting_timer.tick(time.delta());
    }
}
```

Фиг. 3.41 Стреляне на защитната кула

3.6.3 Подобряване на статистиките на защитните кули.

Подобряването на статистиките на защитните кули е реализирано по подобен начин на създаването на кулите. Първоначално се проверява дали играчът в момента не поставя кула и ако поставя, да се блокира от натискането на вече поставена кула. В противен случай се чете всяко

натискане на левия бутон на мишката и ако позицията на мишката се намира върху поставена кула, тя се селектира (фиг. 3.42).

```
fn tower_click() {  
    // If player isn't placing a tower  
    if query.is_empty() {  
        let window = windows.get_primary().unwrap();  
        let (camera, camera_transform) = camera_query.single();  
  
        if mouse.just_pressed(MouseButton::Left) {  
            mouse_click_interaction(&mut commands, window, camera,  
camera_transform, &mut meshes,  
                                &mut materials, &mut clicked_tower, &mut  
towers);  
        }  
    }  
}
```

Фиг. 3.42 Четене на позицията на мишката и потребителския вход

Във функцията `mouse_click_interaction()` се създава графичен интерфейс, чрез който потребителят ще може да избира какво да прави със селектираната кула – дали да я продаде, дали да ѝ смени приоритета на атака или да ѝ подобри статистиките.

В зависимост от вида бутон или клавиш от клавиатурата, който ще натисне играчът, системата `tower_ui_interaction()` ще изпълни съответната команда.

За да се подобрят статистиките на кулата, играчът трябва да избере една от трите пътеки от подобрения чрез потребителския вход. След като избере кое подобрение ще закупи се проверяват парите му и ако има достатъчно се закупува следващото подобрение на избраната пътека (фиг. 3.43).

```

fn tower_ui_interaction () {
    let mut player = player.single_mut();

    for (entity, mut tower, tower_type) in towers.iter_mut() {
        let mut upgrade_path_index: Option<usize> = None;

        // Upgrade tower - Path 1
        if keys.just_pressed(KeyCode::Comma) {
            upgrade_path_index = Some(0);
        }
        // Upgrade tower - Path 2
        else if keys.just_pressed(KeyCode::Period) {
            upgrade_path_index = Some(1);
        }
        // Upgrade tower - Path 3
        else if keys.just_pressed(KeyCode::Slash) {
            upgrade_path_index = Some(2);
        }

        if let Some(path_index) = upgrade_path_index {
            let i = tower.upgrades.upgrades[path_index];
            let upgrades = &upgrades.upgrades[tower_type][path_index];

            if i < upgrades.len() && player.money >= upgrades[i].cost {
                player.money -= upgrades[i].cost;
                tower.upgrade(&upgrades[i], path_index);
            }
        }
    }
}

```

Фиг. 3.43 Избиране на пътека за подобряване на статистиките на кулата

След това във функцията `tower.upgrade()` се осъществява самото подобряване, като се вземат всички статистики от него, които ще бъдат променени, и се прилагат върху сегашните статистики на кулата. И след това се увеличава вътрешният за кулата индекс (`path_index`), който пази индекса на следващото подобряване на пътеката, ако съществува такова (фиг. 3.44).

```

pub fn upgrade(&mut self, upgrade: &Upgrade, path_index: usize) {
    self.total_spent += upgrade.cost as u32;
    for (k, v) in &upgrade.upgrade {
        match *k {
            TowerStat::Damage => {self.damage += *v as u32}
            TowerStat::AttackSpeed => {
                self.attack_speed -= (*v as f32) * 0.01 * self.attack_speed;
                self.shooting_timer.reset();
                self.shooting_timer.set_duration(Duration::from_millis(
                    (1000. * self.attack_speed) as u64));
            }
            TowerStat::Range => {self.range += *v as u32}
        }
    }

    self.upgrades.upgrades[path_index] += 1;
}

```

Фиг. 3.44 Подобряване на статистиките на кула

3.6.4 Продаване и смяна на приоритета на атаката на защитните кули

Продаването и смяната на приоритета на атака на поставени защитни кули също се осъществява от функцията `tower_ui_interaction()`.

При избиране на опцията за продаване се премахва кулата от играта и играчът получава 33% от нейната обща цена (фиг. 3.45).

```

// Sell tower
if keys.just_pressed(KeyCode::Back) {
    commands.entity(entity).despawn_recursive();
    player.money += (tower_type.get_price()/3) as usize;
}

```

Фиг. 3.45 Продаване на кула

При избиране на опцията за промяна на приоритета на атака на кулата се викат функциите `tower.target.prev_target()` или `tower.target.next_target()`, които връщат съответно предишната или следващата опция за приоритет на атаката (фиг. 3.46).

```
// Change targeting priority (left)
else if (keys.pressed(KeyCode::LControl) || keys.pressed(KeyCode::RControl))
    && keys.just_pressed(KeyCode::Tab) {
    tower.target.prev_target();
}
// Change targeting priority (right)
else if keys.just_pressed(KeyCode::Tab) {
    tower.target.next_target();
}
```

Фиг. 3.46 Промяна на приоритета на атака на кула

3.7 Приоритети на атака на кулата

3.7.1 Първи противник

Приоритетът на атака „първи противник“ (first) се насочва към противника, който е най-близко до базата на играча. Избира се най-близкият противник, като се изчислява изминатото от него разстояние по формулата от 3.47.

$$S = v * t \quad (3.47)$$

Противникът има компонент `Movement`, в който се следи разстоянието, което е изминал противникът (`distance_travelled`). То се изчислява по формулата от 3.47 и служи за намиране на най-близкия и най-далечния от базата противник.

Противникът, стигнал най-далече се намира по алгоритъма от фиг. 3.48.

Първоначално се филтрира по разстоянието между кулата и противника. Филтърът взема само тези противници, които са в обхвата на дадената кула. След това с комбинацията от `.max_by_key` и `FloatOrd` се намира противника с най-голямо изминато разстояние. Ако има противник с такива критерии, функцията връща посока към него спрямо кулата, за която се изчислява, като се извади вектора на кулата от вектора на

противника. В противен случай не се връща посока, тъй като няма противници в обхвата на кулата.

```
pub fn first_enemy_direction() -> Option<Vec3> {
    let first_enemy = enemies
        .iter()
        .filter(|(enemy_transform, ..)| {
            Vec3::distance(enemy_transform.translation(),
                           bullet_spawn_pos) <= tower_range as f32
        })
    // Find first enemy that is closest to the base
    .max_by_key(|(.., movement)| {
        FloatOrd(movement.distance_travelled)
    });

    if let Some((first_enemy, ..)) = first_enemy {
        return Option::from(first_enemy.translation() - bullet_spawn_pos); // re-
    }
    return None;
}
```

Фиг. 3.48 Намиране на първия противник

3.7.2 Последен противник

Приоритетът на атака „последен противник“ (last) се насочва към противника, който е най-далече от базата на играча. Избира се най-далечният противник, като се изчислява изминатото от него разстояние.

Алгоритъмът работи по същия начин, по който работи и алгоритъмът за намиране на първия противник, като единствената разлика е, че се сортира по минималното изминато разстояние с `.min_by_key()` вместо с `.max_by_key()`.

3.7.3 Най-близък противник

Приоритетът на атака „най-близък противник“ (close) се насочва към противника, който е най-близко до кулата. Избира се най-близкият противник, като се изчислява разстоянието между него и кулата.

Вместо да взема изминатата от противниците дистанция, тази функция взема разстоянията им от кулата. Филтрира отново с `.min_by_key()` и намира най-малкото разстояние между противника и дадената кула (фиг. 3.49).

```
FloatOrd(Vec3::distance(enemy_transform.translation(), bullet_spawn_pos))
```

Фиг. 3.49 Намиране на най-близкия противник

3.7.4 Най-далечен противник

Приоритетът на атака „най-далечен противник“ (farthest) се насочва към противника, който е най-далече от кулата. Избира се най-далечният противник, като се изчислява разстоянието между него и кулата.

Различава се от приоритета на атака „най-близък противник“ само по филтрирането, вместо с `.min_by_key()`, филтрира с `.max_by_key()`.

3.7.5 Най-силен противник

Приоритетът на атака „най-силен противник“ (strongest) се насочва към противника, който е с най-много кръв. Избира се най-силния противник, като се сравнява броя точки кръв на противниците.

Вместо да взема изминатата от противниците дистанция, тази функция взема броя точки живот на противниците. Филтрира с `.max_by_key()` и намира противника с най-голям брой точки.

3.7.6 Най-слаб противник

Приоритетът на атака „най-слаб противник“ (weakest) се насочва към противника, който е с най-малко кръв. Избира се най-слабият противник, като се сравнява броя точки кръв на противниците.

Различава се от приоритета на атака „най-силен противник“ по това, че филтрира с `.min_by_key()`, вместо с `.max_by_key()`.

3.7.7 Произволен противник

Приоритетът на атака „произволен противник“ (random) се насочва към произволен противник в обхвата на атака на кулата. За реализацията му се използват “rand” и „IteratorRandom” библиотеките (фиг. 3.50).

```
let random_enemy = enemies
  .iter()
  .filter(|(enemy_transform, ..)| {
    Vec3::distance(enemy_transform.translation(),
                   bullet_spawn_pos) <= tower_range as f32
  }).choose(&mut rand::thread_rng());

if let Some((random_enemy, ..)) = random_enemy {
  return Option::from(random_enemy.translation() - bullet_spawn_pos);
}
return None;
```

Фиг. 3.50 Приоритет на атака „произволен противник“

ЧЕТВЪРТА ГЛАВА

РЪКОВОДСТВО ЗА ПОТРЕБИТЕЛЯ

4.1 Изисквания към компютърната конфигурация

4.1.1 Операционна система

Едно от задължителните изисквания към конфигурацията на системата е наличието на Windows или Linux операционна система.

4.1.2 Хардуерни изисквания

Необходимо е компютърната конфигурация да притежава вградена в процесора или външна видео карта, която може да поддържа DirectX 12, Vulkan или в краен случай OpenGL. Потребителят трябва също така да се увери, че има инсталиран съвместим хардуер и драйвери.

От системата на потребителя също така се изисква да има минимум 200 MB свободна памет на твърдия диск, за да инсталира играта. Също така трябва да разполага с поне 300 MB освободена RAM (Random-access memory) памет.

4.2 Инсталация на играта

Инсталацията на играта се осъществява посредством архив с файлов формат „.zip”, намиращ се в „installation“ директорията. Този файл съдържа изпълним файл с разширение „.exe”, чрез който се стартира играта, и папка „assets”, която съдържа всички графични модели, използвани в играта, библиотеки и други данни. След като се разархивира архивът, играта се стартира от изпълнимия файл, който се намира в основната директория.

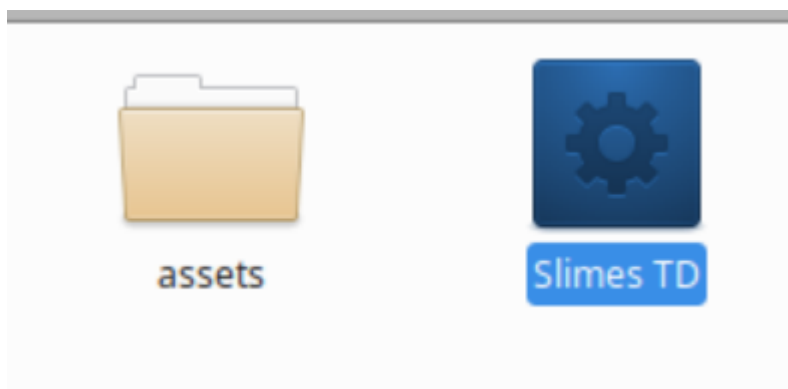
4.3 Стартиране на играта

Играта се стартира, като потребителя изпълни (натисне 2 пъти с курсора на мишката) изпълнимия файл в директорията с наименованието „Slimes TD”. На операционната система Windows файлът е изпълним файл от тип „.exe” (фиг. 4.1).

Name	Date modified	Type	Size
assets	2/27/2023 9:29 PM	File folder	
Slimes TD.exe	3/5/2023 12:50 AM	Application	62,577 KB

Фиг. 4.1 Стартиране на играта на операционната система Windows

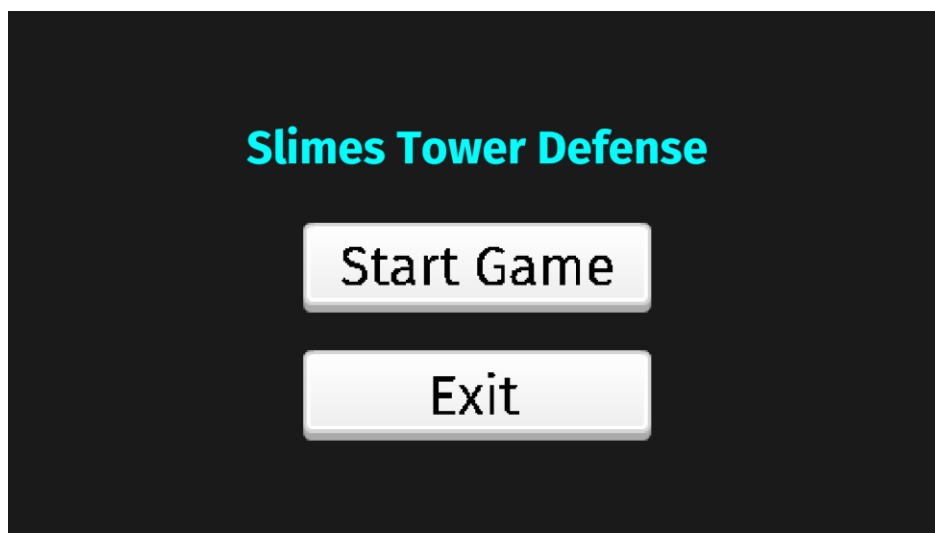
На операционната система Linux, файлът няма тип и се отваря директно и се изпълнява (фиг. 4.2).



Фиг. 4.2 Стартиране на играта от операционната система Linux

4.4 Потребителски вход и как се играе

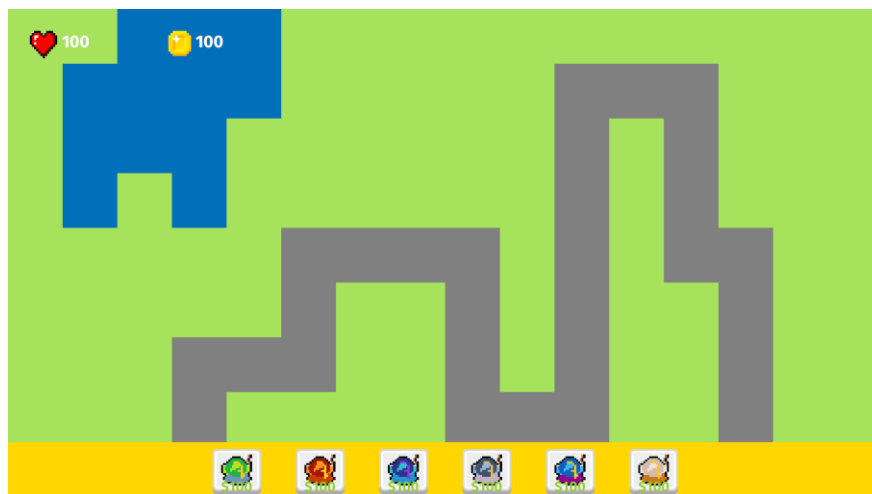
Играта се стартира в главното меню. В главното меню пред играча се изобразява името на играта и 2 бутона – за започване на играта „Start Game” отгоре и за изключване на приложението „Exit” отдолу (фиг. 4.3).



Фиг. 4.3 Началното меню на играта

За да започне играта играчът трябва да натисне бутонът „Start Game”.

След като започне играта пред играчът ще се покаже избраната карта както и 2 графични интерфейса (фиг. 4.4).



Фиг. 4.4 Екран при стартиране на ниво на играта

Първият графичен интерфейс се намира от долната страна на екрана, където са бутоните за изграждане на кули (фиг. 4.5). А другият се намира в горната лява част на екрана, където ще се изобразяват настоящите пари, както и точки живот на играча (фиг. 4.6).



Фиг. 4.5 Графичен интерфейс за различните видове кули



Фиг. 4.6 Графичен интерфейс за парите и точките живот на играча

Когато започне играта започват и първата вълна от противници по дадената пътека. Играчът трябва да им попречи да стигнат до края на пътеката, тъй като ще му вземат точки живот и ще загуби играта, когато точките му живот станат 0. Противниците имат различен брой точки на живот и някои може да са по-трудни за убиване от други.

За да се защити играчът от противниците той трябва да постави защитни кули. Преди да може да постави защитна кула, играчът трябва да има достатъчно пари, за да може да я купи. Цената на всяка кула се изобразява от долната част на всеки бутон (фиг. 4.7).



Фиг. 4.7 Бутон за построяване на кула, изобразяващ цената ѝ

Ако играчът няма достатъчно пари, бутонът за кулата ще се изобрази в по-тъмна сива шарка (фиг. 4.8).



Фиг. 4.8 Затъмнен бутон на кула

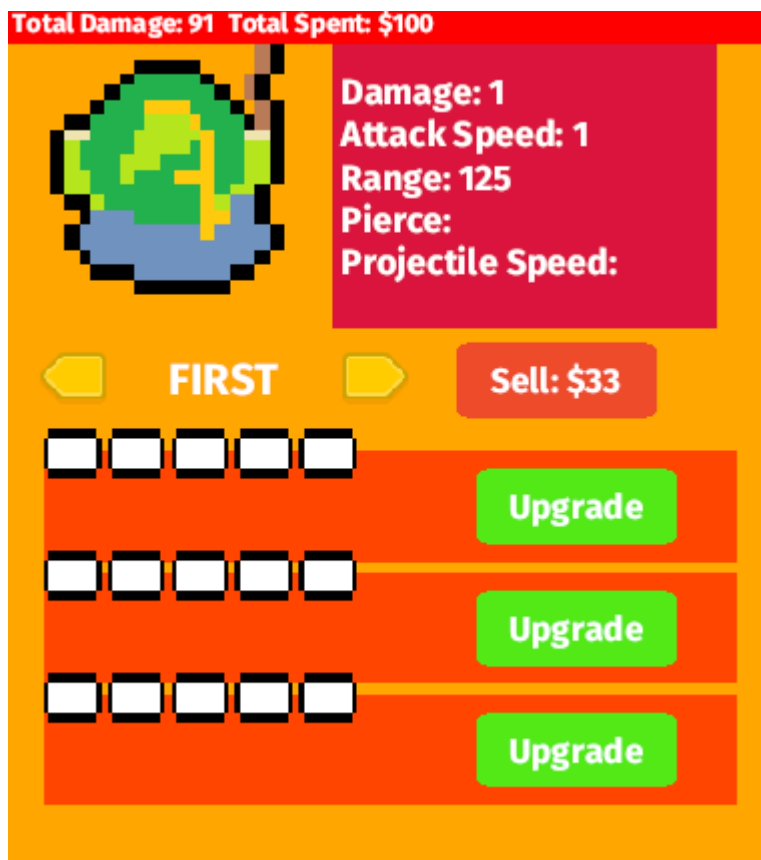
Има 2 начина да се поставят защитни кули – чрез потребителски вход от мишката и чрез клавиатурата.

Чрез мишката той може да постави курсора си върху някой от бутоните с някой вид кула и когато го натисне, малък модел на кулата, заедно с кръг, изобразяващ нейния обхват на атака, ще започне да следва курсора на мишката му. Когато това стане играчът има 2 опции – той може да я постави някъде върху картата с ляв бутон на мишката, или да се откаже от поставянето като натисне десен бутон на мишката.

Чрез другия начин – чрез клавиатурата, играчът може по същия начин да поставя кули. Вместо да натиска бутоните на избрания от него тип кула, той може да натисне съответния клавиш от клавиатурата, който съответства на индекса на кулата. Тоест, ако той иска да построи първата кула, трябва да натисне клавиша с числото „1“. След това ще започне да следва мишката му модел на избраната кула, както и кръг, изобразяващ нейния обхват на атака, и той може да постави кулата чрез кликването на левия бутон на мишката.

Когато бъде поставена някоя кула ще се изобрази от дясната страна на екрана графичен интерфейс със статистики за кулата, бутони за продажба, за смяна на приоритета на атака и бутони за подобряване на статистиките на кулите (фиг. 4.9). Този интерфейс може да се затваря като

се натисне някъде на картата, където няма поставена кула и съответно може да се отвори отново, когато се натисне върху някоя кула на картата.

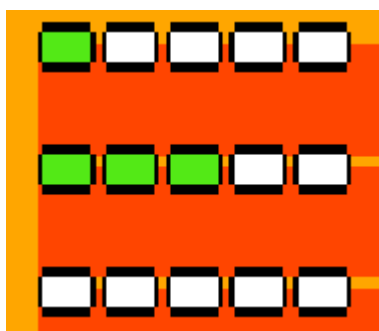


Фиг. 4.9 Интерфейс на кулата

В този интерфейс най-отгоре могат да се видят общи статистики за кулата – колко щета е направила върху противниците и колко пари са похарчени общо за построяването на кулата. Под тези данни, от лявата страна се изобразява иконка на кулата, а от дясната страна – настоящите статистики на кулата – точки на атаката, скорост на атаката, обхват на атаката и други. В средата на интерфейса се показват 3 бутона. От лявата страна се намират 2 бутона, между които има текст. Текстът показва приоритета на атака на кулата, а чрез бутоните той може да се промени. Има общо 7 приоритета на атака подредени в следния ред: „First“, „Last“,

„Close“, „Far“, „Strong“, „Weak“ и „Random“. Чрез левия бутон се взема предишния приоритет на атака, а чрез десния се взема следващия приоритет на атака. От дясно на приоритета на атака има червен бутон за продажба на кулата. В текста му пише колко пари ще получи играчът, когато продаде кулата и съответно ако го натисне ще получи парите и кулата ще изчезне от полето.

В най-долната страна на това меню са изобразени 3 пътеки от подобрения на статистиките на кулата. Трите отделни пътеки си имат отделен зелен бутон с име „Upgrade“ и когато потребителят го натисне и има достатъчно пари, той ще закупи подобрението. Когато го закупи ще види промени в статистиките на кулата, както и във визуалния обхват на кулата, която е избрал. Също така ще може да гледа колко подобрения е закупил от всяка пътека чрез черно-белите квадратчета. Те изобразяват колко общо подобрения има в пътеката и се оцветяват в зелено, когато играчът закупи подобрение (фиг. 4.10).



Фиг. 4.10 Визуални промени при закупуване на подобрение за кулата.

Играчът също има опцията да ползва и потребителски вход от клавиатурата, вместо да натиска бутоните от този интерфейс. С клавиша „Tab“, той може да промени приоритета на атака към следващия приоритет на атака, а с комбинацията от клавиши „Ctrl + Tab“, той може да промени

приоритета на атака към предишния приоритет на атака. С клавиша „Backspace”, той може да продаде кулата и с клавишите „,“, „. “ и „/“, той може да закупи подобрене от съответната пътека, като „,“, „. “ съответства на първата пътека най-отгоре на потребителския интерфейс, „. “ съответства на втората или средната и „/“ съответства на последната.

ЗАКЛЮЧЕНИЕ

Настоящата дипломна работа успешно реализира софтуерен продукт, разработен на игровия двигател Bevy, с езика за програмиране Rust. Проектът реализира всички функционални изисквания, описани в заданието. Реализирана е игра от тип „защитна кула“, където играчът има широк избор от защитни кули, които да използва, за да противодейства на атакуващите го противници. Структурата на играта позволява лесно модифициране на кода и добавяне на нови функционалности за в бъдеще. Играта е разработена да приема потребителски вход от мишката и клавиатурата на потребителя.

При бъдещо развитие на проекта може да се добавят повече видове кули и противници, да се подобри графичния интерфейс на играта, да се добавят нови режими и карти. В бъдещето на софтуерни продукт е предвидено и добавянето на визуални ефекти, както и аудио ефекти, които да подобрят изживяването на играча.

За да може да се играе многократно играта може да предлага ежедневни предизвикателства, които може да са по-сложни и да предизвикват размисъл и дълбоки стратегии от играча.

Също така може да бъде качена онлайн, като може да се добави режим, в който няколко хора могат да играят заедно. В режимите за няколко хора да има режими, в които играчите да си помагат един на друг и да защитават общата си база и също така да има режим, в които те се състезават един срещу друг, за да защитят базата си максимално най-дълго време.

ИЗПОЛЗВАНА ЛИТЕРАТУРА

1. <https://gameranx.com/features/id/13529/article/best-tower-defense-games/>
2. https://web.archive.org/web/20130804175159/http://www.gamasutra.com/view/feature/1728/slamdance_postcolumbine_.php
3. https://web.archive.org/web/20121014094602/http://www.gamasutra.com/view/feature/3722/interview_soren_johnson_spores_.php?page=5
4. <https://www.ign.com/articles/2012/09/24/why-we-love-tower-defense>
5. <https://www.polygon.com/features/2013/8/15/4528228/missile-command-dave-theurer>
6. <https://worldofspectrum.org/software?id=0003639>
7. <https://web.archive.org/web/20140203062250/http://palgn.com.au/11898/tower-defense-bringing-the-genre-back/>
8. https://books.google.com/books?id=LVc1QNGo_g0C&pg=PA90
9. Bob Bates. Game Developer's Market Guide, p. 141, Thomson Course Technology, 2003
10. https://web.archive.org/web/20090131011206/http://www.gamespot.com/gamespot/features/all/real_time/p2_02.html
11. <http://ign.com/lists/top-100-rpgs/2>
12. <http://www.pcgamer.com/why-dungeon-keeper-has-never-been-beaten/>
13. <https://www.ign.com/articles/1998/12/12/turok-2-seeds-of-evil-2>
14. https://web.archive.org/web/20130521133155/http://www.gamasutra.com/php-bin/news_index.php?story=18863
15. https://web.archive.org/web/20130828234150/http://jayisgames.com/archives/2007/01/flash_element_td.php
16. <https://web.archive.org/web/20080726085126/http://www.liberation.fr/actualite/ecrans/290440.FR.php>

17. https://web.archive.org/web/20121015090411/http://www.gamasutra.com/php-bin/news_index.php?story=21102#.UiIErD_3O5I
18. <https://web.archive.org/web/20150915061403/http://gamereporter.uol.com.br/antbuster-game-nacional-online/>
19. <https://web.archive.org/web/20071231004154/https://papodehomem.com.br/antbuster-jogo-brasileiro-premiado-em-disputa-internacional/>
20. https://web.archive.org/web/20121023082025/http://www.gamasutra.com/php-bin/news_index.php?story=19009#.UiIHMz_3O5I
21. <https://web.archive.org/web/20120813195707/http://icrontic.com/article/plants-vs-zombies-nominated-for-pc-game-of-the-year>
22. <http://www.ign.com/articles/2009/04/02/iron-grip-warlord-review>
23. <https://web.archive.org/web/20120103175813/http://www.thisisxbox.com/Arcade/dunge-on-defenders-exceeds-more-than-a-quarter-of-a-million-in-sales/>
24. https://bloons.fandom.com/wiki/Bloons_TD_6
25. <https://bloons.fandom.com/wiki/Maps>
26. https://bloons.fandom.com/wiki/Difficulty#Bloons_TD_6
27. <https://bloons.fandom.com/wiki/Heroes>
28. <https://bloons.fandom.com/wiki/Towers>
29. <https://bloons.fandom.com/wiki/Paragons>
30. [https://bloons.fandom.com/wiki/Monkey_Knowledge_\(BTD6\)](https://bloons.fandom.com/wiki/Monkey_Knowledge_(BTD6))
31. [https://bloons.fandom.com/wiki/Co-Op_Mode_\(BTD6\)](https://bloons.fandom.com/wiki/Co-Op_Mode_(BTD6))
32. https://bloons.fandom.com/wiki/Challenge_Editor
33. https://tds.fandom.com/wiki/Tower_Defense_Simulator_Wiki
34. <https://tds.fandom.com/wiki/Gamemodes>

35. <https://tds.fandom.com/wiki/Towers>
36. <https://tds.fandom.com/wiki/Units>
37. <https://web.archive.org/web/20161117072414/http://www.ign.com/articles/2011/04/14/gemcraft-the-crown-jewel-of-flash-gaming-arrives-on-ios>
38. https://store.steampowered.com/app/1106530/GemCraft_Frostborn_Wrath
39. https://gemcraft.fandom.com/wiki/GemCraft_Lost_Chapter:_Frostborn_Wrath
40. https://rogue-tower.fandom.com/wiki/Rogue_Tower_Wiki
41. https://rogue-tower.fandom.com/wiki/Upgrade_Cards
42. https://rogue-tower.fandom.com/wiki/Map_features
43. https://rogue-tower.fandom.com/wiki/Permanent_Upgrades
44. <https://www.wired.com/2012/05/ff-unreal4/>
45. <https://www.gamesindustry.biz/epic-games-announces-unreal-engine-5-with-first-ps5-footage>
46. <https://venturebeat.com/business/how-epic-games-is-tailoring-unreal-engine-5-to-make-next-gen-graphics-shine/>
47. <https://arstechnica.com/gaming/2020/05/behind-the-scenes-of-that-incredible-unreal-engine-5-tech-demo/>
48. <https://www.unrealengine.com/>
49. <https://www.rockpapershotgun.com/fortnites-jessen-talks-minecraft-pc-gaming-ue4>
50. <https://arstechnica.com/gaming/2014/03/unreal-engine-4-now-available-as-19month-subscription-with-5-royalty/>
51. <https://www.gamesindustry.biz/what-is-the-best-game-engine-is-unity-the-right-game-engine-for-you>
52. <https://www.theverge.com/2017/6/30/15899446/unity-cinemachine-unite-europe-2017-animation>

53. <https://venturebeat.com/pc-gaming/unitys-asset-store-boss-has-big-plans-to-fight-epics-unreal/>
54. <https://www.ft.com/content/f77b7979-c943-4b9d-b7b7-7953b63bea7e>
55. <https://godotengine.org/article/first-public-release/>
56. https://docs.godotengine.org/en/3.5/tutorials/scripting/visual_script/index.html
57. <https://docs.godotengine.org/en/3.5/tutorials/scripting/index.html>
58. https://docs.godotengine.org/en/stable/tutorials/physics/physics_introduction.html
59. <https://docs.godotengine.org/en/stable/tutorials/animation/introduction.html>
60. https://web.archive.org/web/20081115062536/http://www.crytek.com/news/news/?tx_ttnews%5Bpointer%5D=17&tx_ttnews%5Btt_news%5D=76&tx_ttnews%5BbackPid%5D=9&cHash=e4dea47a80
61. <https://www.geeksforgeeks.org/unreal-vs-cryengine/>
62. <https://bevyengine.org/>
63. <https://bevy-cheatbook.github.io/>
64. <https://visualstudio.microsoft.com/vs/>
65. <https://www.infoq.com/news/2014/09/jetbrains-clion/>
66. <https://www.jetbrains.com/clion/features/supported-languages.html>
67. <http://gameprogrammingpatterns.com/game-loop.html>
68. <https://www.gamedev.net/tutorials/programming/general-and-gameplay-programming/collision-detection-r735/>
69. <https://www.kodeco.com/2614-collision-detection-tutorial-for-games>
70. <https://bevy-cheatbook.github.io/programming/events.html>
71. <https://docs.rs/bevy/0.9.1/bevy/ecs/bundle/trait.Bundle.html#implementors>
72. <https://www.rust-lang.org/learn/get-started>

СЪДЪРЖАНИЕ

УВОД.....	1
ПЪРВА ГЛАВА.....	3
1.1 История на игри от типа „защитна кула“	3
1.1.1 Ранна история – аркадни игри	3
1.1.2 Периодът от 2007 до днес.....	4
1.2 Преглед и проучване на други съвременни игри от жанра.....	5
1.2.1 Bloons TD 6	5
1.2.2 Tower Defense Simulator в Roblox	10
1.2.3 GemCraft	13
1.2.4 Rogue Tower.....	14
1.3 Методи за създаване на видео игри.....	15
1.3.1 Разработка на игра с игрови двигател	15
1.3.2 Разработка на игра с персонализиран игрови двигател.....	16
1.3.3 Разработка на игра без игрови двигател.....	16
1.3.4 Разработка на игра с комплекти за разработка на игри	17
1.3.5 Хибриден подход	17
1.4 Съществуващи софтуерни продукти и технологии за създаване на видео игри.....	18
1.4.1 Unreal Engine.....	18
1.4.2 Unity	20
1.4.3 Godot.....	21
1.4.4 CryEngine.....	22
1.4.5 Bevy	23
1.5 Среди за създаване на видео игри	24
1.5.1 Visual Studio.....	24
1.5.2 CLion.....	24
ВТОРА ГЛАВА	25

2.1	Функционални изисквания към софтуерния продукт	25
2.2	Избрани софтуерни средства	26
2.2.1	Игрови двигател.....	26
2.2.2	Интегрирана среда за разработка.....	29
2.3	Алгоритми, използвани при разработването на видео игри.....	30
2.3.1	Цикъл на играта (Game loop)	30
2.3.2	Засичане на сблъсъци/колизии (Collision detection).....	32
2.3.3	Алгоритми за намиране на пътя (Pathfinding).....	32
2.3.4	Управление на вълни от противници.....	33
2.3.5	Приоритет на атака	33
2.3.6	Изкуствен интелект	33
ТРЕТА	ГЛАВА.....	34
3.1	Конструктор на приложението	34
3.2	Главно меню.....	36
3.3	Създаване на компонент за играч и компонент за база	38
3.3.1	Създаване на играч.....	38
3.3.2	Създаване на база.....	39
3.4	Карта.....	40
3.4.1	Камера	40
3.4.2	Зареждане на картата.....	41
3.4.3	Актуализация на пътя на противниците	42
3.5	Противници.....	45
3.5.1	Компонента за противници	47
3.5.2	Типове противници	47
3.5.3	Анимация на модела на противниците	48
3.5.4	Вълни от противници	49
3.6	Защитни кули	54
3.6.1	Поставяне на защитни кули	54
3.6.2	Стреляне на защитните кули	59

3.6.3 Подобряване на статистиките на защитните кули	60
3.6.4 Продаване и смяна на приоритета на атаката на защитните кули	63
3.7 Приоритети на атака на кулата	64
3.7.1 Първи противник	64
3.7.2 Последен противник.....	65
3.7.3 Най-близък противник	65
3.7.4 Най-далечен противник	66
3.7.5 Най-силен противник	66
3.7.6 Най-слаб противник	66
3.7.7 Произволен противник.....	67
ЧЕТВЪРТА ГЛАВА.....	68
4.1 Изисквания към компютърната конфигурация	68
4.1.1 Операционна система.....	68
4.1.2 Хардуерни изисквания	68
4.2 Инсталация на играта.....	68
4.3 Стартиране на играта	69
4.4 Потребителски вход и как се играе	69
ЗАКЛЮЧЕНИЕ	76
ИЗПОЛЗВАНА ЛИТЕРАТУРА	77